Reinforcement Learning for Quantitative Trading: From High-Frequency Trading to Formulaic Alpha Mining

Qin Molei

College of Computing and Data Science

A thesis submitted to the Nanyang Technological University
in partial fulfillment of the requirements for the degree of
Doctor of Philosophy

2026

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research, is free of plagiarised materials, and has not been submitted for a higher degree to any other University or Institution.

Disclosure on AI Use. I declare that ChatGPT was utilised to enhance the clarity and readability of this thesis. I confirm that I have adhered to NTU's Policy on the Use of Generative AI in Research. I have reviewed and edited the content as necessary after using this tool and accept full responsibility for the content of this thesis.

May 2026

.....

Date

.....

Qin Molei

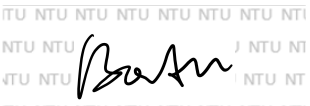
Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice.

May 2026

.....

Date



.....

Prof. Bo An

Authorship Attribution Statement

This thesis contains material from three research papers of which I am the first author. Two of these papers have been accepted and published at peer-reviewed international conferences; the third is a manuscript in preparation for submission.

The technical chapters of this thesis are based on the following papers, with the author list and per-author contributions given in full for each.

Chapter 4 is published as M. Qin, S. Sun, W. Zhang, H. Xia, X. Wang, and B. An, "EarnHFT: Efficient Hierarchical Reinforcement Learning for High Frequency Trading," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI'24)*, vol. 38, no. 13, pp. 14669–14676, 2024.

The contributions of the co-authors are as follows:

- Prof. Bo An proposed the project direction and revised the manuscript.
- Shuo Sun provided guidance on the problem formulation and revised the manuscript.
- I proposed the algorithm, designed and conducted the experiments, and wrote the manuscript.
- Wentao Zhang and Haochong Xia helped with the implementation and the experiments.
- Xinrun Wang provided insightful comments and revised the manuscript.

Chapter 5 is published as M. Qin, X. Cai, Y. Li, H. Xia, C. Zong, S. Sun, X. Wang, and B. An, "FineFT: Efficient and Risk-Aware Ensemble Reinforcement Learning for Futures Trading," in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD'26)*, 2026.

The contributions of the co-authors are as follows:

- Prof. Bo An proposed the project direction and revised the manuscript.
- I proposed the problem, designed the algorithm, conducted the experiments, and wrote the manuscript.
- Xinyu Cai and Yewen Li helped with the implementation and the baseline experiments.
- Haochong Xia and Chuqiao Zong helped with the experiments and revised the manuscript.

- Shuo Sun and Xinrun Wang provided insightful comments and carefully revised the manuscript.

Chapter 6 is based on M. Qin, M. Zhu, C. Zong, and B. An, "AlphaQD: Quality-Diversity Reinforcement Learning for Formulaic Alpha Mining," a manuscript in preparation for submission to the *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD'27, Cycle 1)*.

The contributions of the co-authors are as follows:

- Prof. Bo An proposed the project direction and revised the manuscript.
- I proposed the problem and the method, designed and conducted the experiments, and wrote the manuscript.
- Meiying Zhu ran the baseline experiments.
- Chuqiao Zong helped with the implementation and the experiment scripts.

May 2026

Date

Molei Qin

Qin Molei

Acknowledgements

As my PhD journey at Nanyang Technological University comes to a close, I would like to express my sincere gratitude to the people who have taught, helped and supported me throughout these years. The time of my candidature has been an unforgettable period of growth, exploration and collaboration, and I am fortunate to have worked with people who have consistently pushed me to be more curious, more rigorous and more honest about my own ideas.

First and foremost, I would like to express my deepest gratitude to my supervisor, **Prof. Bo An**, for offering me the opportunity to pursue my PhD under his supervision. Prof. An's patience, sharp judgement and high standards have been invaluable in shaping how I think about research. He has taught me to look for the simplest structure behind a difficult problem, to insist on solid motivation and real-world impact, and to write and present my work with clarity. As my research role model, I have learned from him not only how to conduct research but also a sense of responsibility, professionalism and persistence that I hope to carry into the rest of my career. I am grateful for his guidance throughout this journey and for the AMI research family that he has built and continues to nurture at NTU.

I would also like to thank my Thesis Advisory Committee members, Assoc. Prof. **Long Cheng** (College of Computing and Data Science) and Assoc. Prof. **Tao Chen** (Nanyang Business School, Division of Banking and Finance), for the time, the feedback and the perspective they have generously offered during the milestones of this candidature. Their questions and suggestions helped me sharpen the scope of the thesis and strengthen the technical depth of my work.

This thesis is the result of close collaboration, and I would like to acknowledge the co-authors who shaped the three works it is built on. I owe a particular debt to **Shuo Sun**, my senior in the AMI group, who first introduced me to reinforcement learning for quantitative trading, set an example of rigorous and deployment-minded research, and was a co-author on both EarnHFT and FineFT. I am also

deeply grateful to **Prof. Xinrun Wang**, whose insight and steady mentorship have been a constant throughout the projects that make up this thesis. For EarnHFT, I thank **Wentao Zhang** and **Haochong Xia** for their contributions to the framework, the implementation and the experiments. For FineFT, I thank **Xinyu Cai**, **Yewen Li**, **Haochong Xia** and **Chuqiao Zong** for the technical discussions, the implementation help and the careful reading of the manuscript at every stage. Working with each of them has made the research substantially better than I could have made it alone.

I would also like to thank the broader members of the AMI group at NTU—Xinrun Wang, Rundong Wang, Wei Qiu, Jakub Cerny, Renchunzi Xie, Yanchen Deng, Shuxin Li, Wanqi Xue, Xinyu Cai, Shuo Sun, Pengdeng Li, Hongxin Wei, Pengjie Gu, Yewen Li, Chaojie Wang, Zhenghai Xue, Longtao Zheng, Chuqiao Zong, Sheng Zang, Ying Zheng, Dong Xing, Shanqi Liu, Wentao Zhang, Weihao Tan, Ruhao Jiang, Yuzhou Cao, Zhenxing Ge, Penghui Yang, Qi Wei, Jiacheng Xu, Fuxiang Zhang, Shengming Wang, Jianing Ju, Haochong Xia and Yao Long Teng—for the daily exchange that sustains research: the late-night deadlines, the rebuttal sessions, the group dinners and the everyday discussions in the office. It has been an amazing experience to work in this group, and I have learned a great deal from each of you.

Finally, but most importantly, I would like to thank my parents, my family and my close friends for their unwavering support, patience and love throughout this journey. Their encouragement, from near and far, has carried me through the difficult moments; without it, this thesis would not have been possible. This thesis is dedicated to them.

To my family.

Abstract

With a global capital of over 90 trillion US dollars, financial markets attract the attention of innumerable investors around the world. Quantitative trading converts market data into trading decisions and is, at its core, a sequential decision-making problem under uncertainty, which makes reinforcement learning (RL) a natural modelling tool. Recently, RL has emerged as a promising direction for financial decision making owing to its ability to optimise a long-horizon return rather than a one-step prediction. However, financial markets violate the assumptions on which standard RL rests: they are non-stationary, their reward signals carry a low signal-to-noise ratio, the decision horizons can be extremely long, and a single mistake can be financially catastrophic. Significant effort is therefore still required to close the gap between RL's promise and its real-world deployment in quantitative trading.

This doctoral thesis is devoted to closing that gap by systematically studying what reinforcement learning can do for quantitative trading. We sweep the trading-frequency spectrum, from second-level trading to daily signal generation. It is organised around three tasks: second-level high-frequency trading of a single spot asset, minute-level trading of leveraged cryptocurrency futures, and daily-frequency formulaic alpha mining over a full cross-section of equities. For each task, we identify the binding obstacle that prevents off-the-shelf RL from succeeding and propose a method that addresses it.

First, we study the problem of efficient and adaptive RL for high-frequency trading, where a single second-level agent evaluated over weeks faces a horizon on the order of one million steps and where rapid regime changes break any monolithic policy. We propose EarnHFT, a three-stage hierarchical RL framework with three components: (i) a Q-teacher computed by dynamic programming over future prices, which accelerates and stabilises second-level training while preserving the optimal Bellman fixed point of the supervised update; (ii) a pool of trend-specialised agents trained under different preference parameters and selected by profitability;

and (iii) a minute-level router that picks a specialist from the pool as market conditions change. Extensive experiments on four cryptocurrency markets demonstrate that EarnHFT exceeds the strongest of six baselines by at least 30% in profitability across all four datasets.

We then investigate the problem of risk-aware ensemble RL for leveraged futures, where high leverage amplifies the aleatoric noise of the reward and where the absence of any awareness of capability boundary exposes existing methods to liquidation during black-swan events. We propose FineFT, a three-stage ensemble RL framework including: (i) a selective-update mechanism, driven by an ensemble temporal-difference matrix, that trains diverse specialists efficiently and segments the market automatically; (ii) variational autoencoders, one per market dynamic, that capture each specialist's capability boundary and detect out-of-distribution market states; and (iii) a risk-aware heuristic router that routes between specialists and a conservative fallback by VAE losses on recent market states. Extensive experiments on four cryptocurrency futures markets demonstrate that FineFT reduces risk—measured by maximum drawdown—by more than 40% relative to the runner-up among twelve baselines while remaining the most profitable learned method.

Third, we turn from execution to signal generation and study the problem of formulaic alpha mining, where pool-based RL miners couple the per-alpha reward to an evolving pool—a boosting-style objective that trains the policy against a moving target and offers no a-priori generalisation bound. We propose AlphaQD, a three-stage quality-diversity RL method including: (i) an efficient relational-graph GFlowNet generator trained by trajectory balance against an archive-independent terminal reward; (ii) a Return-Source MAP-Elites archive whose behaviour descriptor is the normalised daily information-coefficient series of each formula, an empirical proxy for the formula's return source; and (iii) a top-k cell-selection rule that exports one high-quality representative per occupied cell as the deployed pool. Extensive experiments on full-market Chinese A-share and U.S. benchmarks demonstrate that AlphaQD lies in the leading signal-quality cluster on both markets and attains the highest annualised return and the strongest net Sharpe among learned formula-mining methods on A-shares, with the smallest seed-to-seed variance and the lowest turnover among learned pool-style methods.

Read together, the three works share a single methodological skeleton—decompose the non-stationary market into approximately stationary sub-problems, specialise a learner to each, and aggregate the specialists through a mechanism suited to the task—and trace a deliberate progression in how that aggregation is performed and how far it is decoupled from the policy's learning reward: from a learned online RL router that selects a single specialist in EarnHFT, through a generative risk-aware heuristic router in FineFT, to a fully structural quality-diversity archive with a ridge combiner in AlphaQD that is kept entirely outside the learning reward. The thesis therefore offers both a set of state-of-the-art methods for distinct quantitative-trading tasks and a unifying perspective on how reinforcement learning should be structured for non-stationary financial markets.

Contents

List of Figures

List of Tables

Symbols and Acronyms

Symbols

$\mathcal{S},\ \mathcal{A}$	the state space and the action space of an MDP
π_θ	a policy parameterised by θ
$Q(s,a)$	the action-value (Q) function
$Q^*(s,a)$	the optimal action-value function
γ	the discount factor
r_t	the reward received at time t
V_t	the net value (margin balance) of the account at time t
$H_t,\ L_t$	the position and the leverage at time t
$\mathcal{H}(\,\cdot\,)$	the Huber loss
$\mathrm{KL}(\,\cdot\,\|\,\cdot\,)$	the Kullback–Leibler divergence
α	a formulaic alpha (symbolic expression)
$\mathrm{IC}_t(\alpha)$	the daily cross-sectional information coefficient at time t
$\mathrm{ICIR}(\alpha)$	the information ratio of the IC series
$\mathbf{r}(\alpha)$	the centred, unit-normalised IC series (return-source descriptor)
β	a preference (EarnHFT) or reward-temperature (AlphaQD) coefficient

Acronyms

RL	Reinforcement Learning
HFT	High-Frequency Trading
MDP	Markov Decision Process
DP-MDP	Dynamic Parametric Markov Decision Process
HRL	Hierarchical Reinforcement Learning
DQN	Deep Q-Network
DDQN	Double Deep Q-Network

PPO	Proximal Policy Optimisation
TD	Temporal Difference
ETD	Ensemble Temporal Difference
MoE	Mixture of Experts
VAE	Variational Autoencoder
OOD	Out-of-Distribution
ID	In-Distribution
QD	Quality-Diversity
GFlowNet	Generative Flow Network
TB	Trajectory Balance
RGCN	Relational Graph Convolutional Network
MAP-Elites	Multi-dimensional Archive of Phenotypic Elites
PFS	Perturbation Fidelity Score
PCA	Principal Component Analysis
RPN	Reverse Polish Notation
GP	Genetic Programming
DTW	Dynamic Time Warping
LOB	Limit Order Book
OHLC	Open-High-Low-Close
IC	Information Coefficient
ICIR	Information-Coefficient Information Ratio
MACD	Moving Average Convergence Divergence
IV	Imbalance Volume
TR	Total Return
ASR	Annualised Sharpe Ratio
ACR	Annualised Calmar Ratio
ASoR	Annualised Sortino Ratio
MDD	Maximum Drawdown
AVOL	Annualised Volatility
Crypto	Cryptocurrency

Part I

Preliminaries

Chapter 1

Introduction

1.1 Quantitative Trading and Reinforcement Learning

Quantitative trading is the discipline of converting market data into trading decisions through explicit, data-driven rules. A quantitative strategy ingests a stream of prices, volumes, and order-flow signals, and at each decision point it must choose what to hold and how to adjust that holding. Stated this way, quantitative trading is, at its core, a problem of *sequential decision-making under uncertainty*: the present action changes the portfolio, the portfolio interacts with a stochastic and adversarial market, and the consequences are realised only over a horizon of subsequent steps. The natural formal language for such problems is reinforcement learning (RL), in which an agent learns a policy that maps states to actions so as to maximise a cumulative reward.

Reinforcement learning is attractive for trading for reasons that distinguish it from the supervised learning that dominates much of quantitative finance. Supervised approaches predict a label—a future return, a price direction—under the assumption that training samples are independent and identically distributed (IID), and they optimise a one-step predictive loss that is agnostic to how the prediction is used. Trading violates both premises. Transaction costs and inventory constraints make the profitability of an action depend on the current position, so consecutive

decisions are coupled rather than independent; and the quantity a trader actually cares about is the long-horizon, risk-adjusted profit of a *sequence* of decisions, not the accuracy of any single forecast. Reinforcement learning addresses both directly: it optimises a return that accumulates over the trajectory, and it models the path dependence induced by costs and positions as part of the environment dynamics. Encouraged by RL's successes in games and robotics, a substantial body of work has applied it across the quantitative-trading stack, from order execution and intraday trading to portfolio management [1–3].

The premise of this thesis is therefore that reinforcement learning is the right tool for quantitative trading; its subject is the gap between that premise and practice. Financial markets violate, sometimes severely, the assumptions on which standard RL algorithms were designed and validated. The contribution of this thesis is to identify, for several distinct and economically important trading tasks, exactly which assumption breaks, and to design a method that restores the relevant condition locally, or mitigates its effect, rather than papering over its symptoms.

1.2 Why Reinforcement Learning is Hard in Financial Markets

The difficulties that financial markets pose for reinforcement learning recur across tasks, although each task stresses a different subset of them. It is useful to enumerate them once, because the technical chapters can then be read as attacks on specific items in this list.

- **Non-stationarity.** The defining property of financial markets is that their statistics drift. The relationship between observable features and future returns changes over time as market regimes shift between trends, ranges, and crises. This breaks the stationary-MDP assumption underpinning convergence guarantees for value- and policy-based RL: a policy that is optimal on the training distribution can be actively harmful once the regime changes, because the environment the agent now faces is not the one it was trained on.

- **Low signal-to-noise ratio.** The predictable component of asset returns is small relative to the noise. The reward an RL agent receives is therefore a faint signal buried in stochastic price movement, which slows learning, inflates the variance of gradient estimates, and makes it easy to overfit idiosyncratic patterns of the training window that do not generalise.

- **Extremely long decision horizons.** At high frequency, the number of decision steps over an evaluation period is enormous. A second-level agent assessed over a few weeks faces a horizon on the order of one million steps, against which the few-thousand-step horizons of standard RL benchmarks [4] are small. Long horizons demand far more data to converge [5] and make both training and evaluation computationally heavy.

- **Risk and irreversibility.** A trading loss is real and, under leverage, can be catastrophic: a single excursion into an unfamiliar market state can liquidate an account. Unlike a game, where a poor episode merely lowers the average score, trading has hard, asymmetric downside that an expected-return objective alone does not capture.

- **Reward design and coupling.** What constitutes a good reward is itself non-trivial. When the objective concerns a *set* of strategies or signals—for example, the marginal contribution of a new factor to an existing pool—it is tempting to fold the pool into the reward. Doing so couples the learning target to a quantity that itself evolves during training, silently re-introducing non-stationarity through the reward.

- **Generalisation.** The training window is finite and the future is non-stationary, so a discovered strategy or signal must hold up out of sample. This places a premium on methods whose effective complexity is small relative to the search space, and on selection rules that do not merely fit the training window.

No single method removes all of these obstacles at once, nor should one expect it to: the obstacles trade off against one another, and the binding constraint differs by task. This thesis instead studies three concrete tasks, isolates the obstacle that is decisive for each, and addresses it.

1.3 Tasks, Challenges, and Solutions

The three studies that constitute this thesis are organised along the spectrum of trading frequency and, with it, a progression from *trade execution* to *signal generation*. Table 1.1 summarises them; the remainder of this section introduces each in turn.

TABLE 1.1: The three trading tasks studied in this thesis, the decisive obstacle each presents to standard reinforcement learning, and the method developed in response. The tasks span the frequency spectrum and progress from execution to signal generation.

Chapter	**Task**	**Decisive challenge**	**Method**
Chapter 4 (EarnHFT)	Second-level spot high-frequency trading	Extremely long horizon ($\sim 10^6$ steps) cripples data efficiency; regime changes break any single policy	Three-stage *hierarchical* RL: dynamic-programming Q-teacher, a pool of trend specialists, and a minute-level router
Chapter 5 (FineFT)	Minute-level leveraged futures trading	Leverage amplifies reward stochasticity; agents are unaware of their competence boundary and are liquidated on unseen states	Three-stage *ensemble* RL: selective-update specialists, per-dynamic VAE boundaries, and risk-aware routing with a conservative fallback
Chapter 6 (AlphaQD)	Daily formulaic alpha mining over a full cross-section	Boosting-style rewards couple the target to an evolving pool, causing non-stationarity and offering no generalisation handle	Three-stage *quality-diversity* RL: GFlowNet exploration under an archive-independent reward, MAP-Elites archiving, and top-k cell export

Table 1.2 makes the thesis spine explicit by tracing, across the three studies, what is decomposed, what is specialised, how the specialists are aggregated, and how tightly that aggregation is coupled to the policy's learning target.

Task I: High-frequency trading (Chapter 4). The first task is second-level high-frequency trading (HFT) of a single cryptocurrency, where the agent repeatedly adjusts its position to profit from short-lived price fluctuations. Two obstacles dominate. First, the horizon is extremely long: evaluating a second-level agent over weeks entails roughly one million steps per month, so naive RL is hopelessly data-inefficient. Second, the market is non-stationary at a scale that

TABLE 1.2: The thesis spine across the three studies. The decomposition criterion changes with the task, but the response to non-stationarity is the same—decompose, specialise, and aggregate—and the aggregation mechanism is progressively decoupled from the policy's learning target.

Chapter	**Decomposition criterion**	**Specialist**	**Aggregation mechanism**	**Coupling to policy reward**
Chapter 4 (EarnHFT)	Market trend (slope-based segmentation)	Second-level DDQN agent per trend	Minute-level DDQN router trained by RL on historical data and frozen for deployment	*Tightly coupled*: the aggregation mechanism is itself a reward-maximising policy
Chapter 5 (FineFT)	Latent dynamic of a dynamic-parametric MDP	Selectively-updated learner per dynamic	Heuristic router driven by per-dynamic VAE losses and a conservative fallback	*Partly decoupled*: selection is a generative uncertainty estimate, not a learned policy
Chapter 6 (AlphaQD)	Return-source behaviour (normalised daily IC series)	Per-cell elite of a MAP-Elites archive	Static top-k cell export with a shared ridge combiner	*Fully decoupled*: the generator's reward never reads the archive, and the archive never edits the reward

defeats any single policy, because strategies that profit in a rising market lose in a falling one. EarnHFT resolves both through hierarchy. At high frequency a single trader's orders do not move prices, so future prices are exogenous to the policy. EarnHFT exploits this structural feature: it computes, by dynamic programming over future prices, an optimal action-value teacher that accelerates and stabilises second-level training. It then trains a *pool* of agents, each specialised to a market trend, and a minute-level *router* that selects among them as conditions change. EarnHFT exceeds the strongest baseline by at least 30% in profitability across four markets.

Task II: Leveraged futures (Chapter 5). The second task moves from spot to leveraged futures, introducing long and short positions and high leverage. Leverage changes the nature of the difficulty in two ways. It amplifies the aleatoric noise of the reward, so that nearly identical states under identical actions can yield wildly different outcomes, undermining convergence; and it raises the cost of error to the point of liquidation, which exposes a deficiency that low-leverage methods can ignore—the absence of any *self-awareness of competence*. An agent that acts confidently outside the market states it was trained on can be wiped out by a

single black-swan move. FineFT addresses both. A selective-update mechanism assigns each transition to the learner that already explains it best, together with its neighbours, training a diverse ensemble efficiently. A per-dynamic variational autoencoder then learns the *capability boundary* of each specialist. At deployment, FineFT routes between the specialists and a conservative fallback by detecting when the current market state is out of distribution. The method reduces risk—measured by maximum drawdown—by more than 40% while remaining the most profitable learned baseline.

Task III: Formulaic alpha mining (Chapter 6). The third task departs from execution altogether. Rather than deciding how to trade, it asks an agent to *generate* the predictive signals—short, interpretable, symbolic formulas, or "alphas"—that a downstream portfolio is built upon. Here the binding obstacle is one of reward design. The dominant line of RL alpha miners scores a new formula by its marginal contribution to a running pool, which is a boosting-style objective: the reward changes every time the pool changes, so the policy is trained against a moving target, and the resulting ensemble carries no a-priori generalisation bound. AlphaQD diagnoses this coupling and removes it. The generator is a graph-based GFlowNet trained against a reward that depends only on the candidate and a fixed training window, never on the archive. Diversity is relocated into a separate quality-diversity archive. Its behaviour descriptor is the normalised daily information-coefficient series of each formula, an empirical proxy for the formula's return source. On full-market Chinese A-share and U.S. benchmarks AlphaQD attains the leading signal-quality cluster on both markets with the lowest seed-to-seed variance and turnover among learned pool-style methods.

1.4 A Recurring Methodological Pattern

Although the three studies were motivated by different tasks and developed independently, they share a methodological skeleton that is worth naming, because it organises the thesis. In each case the response to non-stationarity is the same three-move pattern:

1. **Decompose** the non-stationary market into sub-problems that are approximately stationary—by market trend in EarnHFT, by latent dynamic in FineFT, by return-source behaviour in AlphaQD.

2. **Specialise** a learner to each sub-problem, producing a diverse pool of experts rather than a single monolithic policy.

3. **Aggregate** the specialists at deployment through a task-appropriate mechanism—routing to a single specialist when one strategy should own the prevailing regime, or combining several when their signals are complementary.

The thesis does not impose this pattern as dogma; it emerges because decomposition is the most direct way to convert a non-stationary problem into a family of problems on which RL's stationary-MDP assumption approximately holds. What does deserve emphasis is a lesson that surfaces only when the three aggregation mechanisms are compared. The mechanism that composes the specialists—a router that selects one in EarnHFT and FineFT, a structural archive that combines several in AlphaQD—is the component most easily mis-designed, because the natural temptation is to *learn* it jointly with the specialists, or to fold its criterion into the learning reward. Both couplings re-introduce, through the back door, the very non-stationarity the decomposition was meant to remove: a learned router changes the effective environment of the specialists as it learns, and an aggregation-aware reward changes the target as the pool evolves. Across the three chapters the aggregation mechanism is therefore progressively *decoupled* from the policy reward. In EarnHFT the router is itself trained by reinforcement learning against the trading reward, offline on historical data and frozen before deployment. FineFT replaces this with a generative, risk-aware heuristic router that requires no policy learning at all. AlphaQD goes further: the aggregation rule is a structural archive whose membership never enters the generator's reward. This decoupling is the connective thread of the thesis and the single most transferable insight it offers. Stated as a single claim:

> *In a decompose–specialise–aggregate design for a non-stationary environment, the aggregation mechanism should be kept out of the policy's learning target.*

1.5 Contributions

The contributions of this thesis are as follows.

- **EarnHFT** (Chapter 4): the first, to our knowledge, hierarchical reinforcement-learning framework for single-asset high-frequency trading. Its technical core is an optimal action-value teacher computed by dynamic programming over future prices, which is shown both empirically and theoretically to accelerate and stabilise second-level training; together with trend-conditioned specialist training and a minute-level router, it delivers a 30% profitability margin over the strongest baseline.
- **FineFT** (Chapter 5): a risk-aware ensemble framework for leveraged futures. It introduces a selective-update mechanism, driven by an ensemble temporal-difference matrix, that trains diverse specialists efficiently and segments the market automatically; and it is, to our knowledge, the first method to use variational autoencoders for out-of-distribution detection in RL for quantitative trading, turning competence-awareness into an explicit, generative risk control that reduces risk by more than 40%.
- **AlphaQD** (Chapter 6): a quality-diversity reinforcement-learning method for formulaic alpha mining. It diagnoses the boosting-style reward of pool-based miners as the joint source of reward non-stationarity and absent generalisation control, and replaces it with an archive-independent reward and a MAP-Elites archive keyed on a novel return-source behaviour descriptor. The resulting selection rule admits a finite-sample ensemble generalisation bound whose complexity scales with the number of occupied archive cells rather than the cardinality of the formula space.
- **A unifying perspective** (Chapter 1, Chapter 7): the recognition that decompose–specialise–aggregate is a general response to market non-stationarity, and the accompanying lesson that the aggregation mechanism should be decoupled from the policy reward, supported by three increasingly decoupled instantiations.

1.6 Organisation of the Thesis

The thesis is organised into four parts. **Part I** (Preliminaries) comprises this introduction, Chapter 2, which establishes the shared financial and reinforcement-learning background of the technical chapters, and Chapter 3, which surveys related work organised by task and synthesises the open challenges that motivate them. **Part II** (Reinforcement Learning for Trading) contains the two trading studies: Chapter 4 presents EarnHFT for high-frequency trading, and Chapter 5 presents FineFT for leveraged futures. **Part III** (Reinforcement Learning for Signal Generation) contains Chapter 6, which presents AlphaQD for formulaic alpha mining. **Part IV** (Epilogue) consists of Chapter 7, which revisits the unifying perspective, states the limitations of each study, and outlines directions for future work. Supporting proofs and extended derivations are presented within the technical chapters in which they arise.

Chapter 2

Preliminaries

This chapter introduces the shared vocabulary used throughout the thesis. Section 2.1 introduces the financial instruments, market microstructure, and evaluation metrics common to all three studies; the metric definitions given there are referenced rather than repeated in later chapters. Section 2.2 is a brief primer on the reinforcement-learning machinery—from Markov decision processes to GFlowNets and quality-diversity search—used across the thesis. The survey of related work and the open challenges that motivate the technical chapters are deferred to Chapter 3.

2.1 Financial Markets and Quantitative Trading

2.1.1 Instruments and Market Microstructure

The three studies operate on three increasingly complex trading settings. EarnHFT (Chapter 4) trades a single *spot* cryptocurrency, where the asset is bought and sold outright and only long positions are taken. FineFT (Chapter 5) trades *perpetual futures*, standardised contracts that permit both long and short positions, magnify exposure through leverage, and incur a periodic funding fee. AlphaQD (Chapter 6) operates on the *cross-section* of an entire equity market, where the object of interest is the relative ranking of thousands of assets on each day rather than the timing of a single instrument. The following concepts recur throughout.

Limit order book (LOB). The limit order book aggregates all resting buy and sell orders by price level and is the standard description of market microstructure [6]. An m-level LOB at time t is denoted $B_t = (p_t^{b_1}, p_t^{a_1}, q_t^{b_1}, q_t^{a_1}, \ldots, p_t^{b_m}, p_t^{a_m}, q_t^{b_m}, q_t^{a_m})$, where $p_t^{b_i}$ and $p_t^{a_i}$ are the level-i bid and ask prices and $q_t^{b_i}, q_t^{a_i}$ are the corresponding quantities. Figure 2.1 illustrates a snapshot. A *market order* consumes liquidity from successive levels of the book, so its average execution price differs from the best quote—an effect known as slippage that any high-fidelity simulator must model.

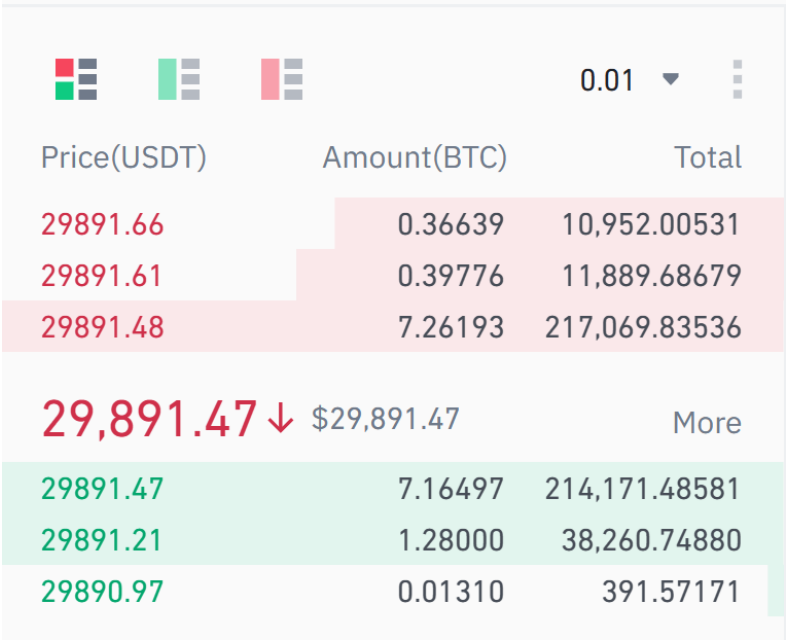


FIGURE 2.1: A snapshot of a multi-level limit order book. Resting bids (buy orders) and asks (sell orders) are aggregated by price level; a market order walks the book from the best quote inward, so larger orders execute at progressively worse prices.

OHLC and trades. Executed transactions are summarised over a time interval by the open, high, low, and close prices, denoted $x_t = (p_t^o, p_t^h, p_t^l, p_t^c)$. The raw stream of executed orders, or trades, records price, quantity, and side. For leveraged contracts, a *mark price* M_t is used as the reference fair value at which the position is valued and against which liquidation is assessed.

Technical indicators. Both microstructure and price-history features are summarised by technical indicators—deterministic functions of recent LOB and trade data designed to expose latent patterns, denoted $y_t = \phi(\cdot)$. Examples are the imbalance volume (IV) [7], which measures buying versus selling pressure in the

book, and the moving-average convergence divergence (MACD) [8], which contrasts trends over different windows. These indicators are inputs to the RL state in all three studies, but as standalone rules they are sensitive to hyperparameters and produce false signals in volatile markets.

Position, leverage, and value. A trader's position H_t is the signed quantity held; under leverage L_t, the same capital supports a larger position and amplifies both gains and losses. The account value—net value for spot, margin balance for futures—is the sum of cash and the marked value of the position, net of realised costs such as commissions and funding fees.

2.1.2 The Trading-Frequency Spectrum

Trading strategies are conventionally classified by the frequency at which they act, and the binding difficulty changes along this spectrum. *High-frequency* strategies act at the second or sub-second scale and profit from fleeting microstructure imbalances; their defining computational challenge is the sheer length of the decision horizon. *Medium-frequency* strategies act at the minute-to-hour scale, where leverage and overnight or funding risk become material. *Low-frequency* strategies act daily or slower; here the difficulty is not the horizon but the discovery of robust predictive signals from a low signal-to-noise cross-section. This thesis traverses the entire spectrum, from second-level execution in Chapter 4 to daily signal generation in Chapter 6.

2.1.3 Evaluation Metrics

Strategies are evaluated at two layers. The *portfolio layer* measures the realised profit-and-loss of an executed strategy; the *signal layer*, used for alpha mining, measures the predictive quality of a signal independently of any portfolio construction. Let $\mathbf{ret} = (ret_1, \ldots, ret_t)$ denote the per-step return series and m the number of steps in a year.

Portfolio-layer metrics. The following six metrics are used throughout Chapter 4 and Chapter 5, and the return/risk pair recurs in Chapter 6.

- **Total Return (TR)**: the overall return over the trading period, $TR = (V_t - V_1)/V_1$, where V_1 and V_t are the initial and final account values.
- **Annualised Volatility (AVOL)**: $\sigma[\mathbf{ret}]\sqrt{m}$, a measure of risk.
- **Maximum Drawdown (MDD)**: the largest peak-to-trough decline of the value curve, capturing the worst case.
- **Annualised Sharpe Ratio (ASR)**: $\mathbb{E}[\mathbf{ret}]/\sigma[\mathbf{ret}]\sqrt{m}$, return per unit of total risk.
- **Annualised Calmar Ratio (ACR)**: $\mathbb{E}[\mathbf{ret}]\, m/\mathrm{MDD}$, return per unit of worst-case loss.
- **Annualised Sortino Ratio (ASoR)**: $\mathbb{E}[\mathbf{ret}]\sqrt{m}/DD$, where DD is the downside deviation; it penalises only harmful volatility.

Signal-layer metrics. For a cross-sectional signal α and an h-day forward return $r^{(h)}$, the daily *information coefficient* is the cross-sectional correlation $\mathrm{IC}_t(\alpha) = \mathrm{corr}(\alpha_t, r_t^{(h)})$. Its time average and stability are combined into the *information ratio* $\mathrm{ICIR}(\alpha) = \overline{\mathrm{IC}(\alpha)}/\hat{\sigma}(\mathrm{IC}(\alpha))$, the canonical scalar quality of a single alpha [9], since it rewards predictive strength and temporal stability simultaneously. The rank-based variant, *RankIC*, replaces Pearson with Spearman correlation and is robust to outliers.

2.2 Reinforcement Learning Foundations

This section is a brief primer, confined to the reinforcement-learning machinery the technical chapters actually use; a reader fluent in RL may skip directly to the task survey of Chapter 3, which carries the weight of the literature review.

2.2.1 Markov Decision Processes

A Markov decision process (MDP) is the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma, T)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ the transition function, r the reward function, $\gamma \in (0, 1]$ the discount factor, and T the horizon. At each step the

agent observes s_t, takes an action a_t under a policy $\pi_\theta : \mathcal{S} \times \mathcal{A} \to [0, 1]$, and receives a reward r_t and a next state s_{t+1}. The objective is to find a policy maximising the expected discounted return $\pi^\star = \arg\max_\theta \mathbb{E}_{\pi_\theta}[\sum_{t=0}^{T} \gamma^t r_t]$. The convergence guarantees of standard RL algorithms presuppose that P and r are *stationary*; the central difficulty of this thesis is that financial environments are not, a fact made explicit in Chapter 5 by the dynamic-parametric MDP formulation that augments the tuple with a slowly evolving latent dynamic.

2.2.2 Value-Based Methods

Value-based methods learn the action-value function $Q(s, a)$, the expected return of taking a in s and acting optimally thereafter. Q-learning updates Q towards the temporal-difference (TD) target $r + \gamma \max_{a'} Q(s', a')$. The Deep Q-Network (DQN) [10] parameterises Q with a neural network and stabilises learning through experience replay and a target network. Double DQN (DDQN) [11] decouples action selection from evaluation to reduce the overestimation bias of the max operator. EarnHFT and FineFT both build on DDQN, augmenting its TD target with supervision from a dynamic-programming teacher; the convergence of this augmented operator is established in Chapter 4.

2.2.3 Policy-Based Methods

Policy-based methods optimise the policy directly. Proximal Policy Optimisation (PPO) [12] maximises a clipped surrogate objective with importance sampling, trading some bias for stability and sample reuse. In trading, policy-based methods are prone to collapsing onto trivial policies—holding the current position—when transaction costs make any action locally unprofitable, a failure mode documented empirically in Chapter 4.

2.2.4 Distributional and Ensemble Reinforcement Learning

Two families address the high stochasticity of financial rewards. *Distributional* RL learns the full return distribution rather than its mean; quantile-regression

DQN [13] and its risk-averse variants represent the return by a set of quantiles and can act on the lower tail to avoid worst cases. *Ensemble* RL trains several learners and aggregates them: averaged DQN reduces target variance, while SUNRISE [14] treats inter-learner disagreement as a measure of uncertainty and down-weights uncertain samples. FineFT belongs to the ensemble family but departs from prior work by updating learners *selectively* rather than uniformly, which is what allows specialists to emerge.

2.2.5 Hierarchical Reinforcement Learning

Hierarchical RL decomposes a long-horizon task into a hierarchy of sub-tasks, typically a high-level policy that selects among low-level policies or temporally extended options. In trading, this naturally maps onto a separation of time scales: a slow policy that reacts to macro conditions and fast policies that execute. Prior hierarchical trading frameworks address portfolio management and order execution [15] or select among strategies by market situation [16]. EarnHFT brings this idea to single-asset high-frequency trading, where the high-level router operates at the minute scale and the low-level specialists at the second scale.

2.2.6 Generative Flow Networks

Generative flow networks (GFlowNets) [17] learn a stochastic policy that constructs compositional objects—here, symbolic expressions—step by step, such that the probability of producing a complete object is proportional to a terminal reward. Trained by the trajectory-balance objective [18], a GFlowNet samples proportionally to reward rather than collapsing onto the single maximiser, so it covers many high-reward modes of the search space under a fixed budget. This mode-covering property is precisely what an alpha-mining search requires, and AlphaQD adopts a relational-graph GFlowNet as its generator.

2.2.7 Quality-Diversity Search and MAP-Elites

Quality-diversity (QD) algorithms seek not a single optimum but an archive of high-quality solutions that are diverse along a chosen *behaviour descriptor*. MAP-Elites discretises the descriptor space into cells and retains the best solution(s) per cell, yielding a structured population that covers the behaviour space. QD has not previously been applied to formulaic alpha mining; AlphaQD's contribution is to identify a meaningful behaviour descriptor for this domain—the normalised daily IC series, an empirical proxy for a formula's return source—and to use the archive as the mechanism that organises diversity and selection outside the learning reward.

2.2.8 Variational Autoencoders and Out-of-Distribution Detection

A variational autoencoder (VAE) [19] learns a probabilistic encoder that maps inputs to a regularised latent distribution and a decoder that reconstructs them. Because it provides a principled, probabilistic measure of how well a new input matches the training distribution, the VAE reconstruction loss is a standard signal for out-of-distribution (OOD) detection. FineFT trains a VAE per market dynamic and uses its loss to detect when the current market state lies outside a specialist's competence, converting OOD detection into an explicit risk control.

Chapter 3

Literature Review

This chapter surveys the prior work on which the three technical chapters build. With the financial and reinforcement-learning preliminaries of Chapter 2 in hand, Section 3.1 reviews reinforcement learning for quantitative trading organised by task—single-asset and intraday trading, high-frequency trading, portfolio management, order execution, strategy routing, and formulaic alpha mining—before turning to the cross-cutting question of uncertainty and non-stationarity. Section 3.2 then synthesises the open challenges that motivate Chapter 4, Chapter 5, and Chapter 6.

3.1 Reinforcement Learning for Quantitative Trading

Reinforcement learning has been applied across the entire quantitative-trading stack, and we survey that body of work organised by task. For each task we note the dominant design and the assumption it leaves unaddressed, which the open challenges of Section 3.2 then collect. Open frameworks and benchmarks have standardised much of this research—FinRL [20], TradeMaster [21], Qlib [22], and the PRUDEX evaluation compass [23]—and the studies in this thesis follow their evaluation conventions where applicable.

3.1.1 Single-Asset and Intraday Trading

The earliest and largest body of RL-for-trading work targets a single asset at the daily-to-intraday scale, where the agent maps a window of price and technical features to a discrete trading action. Foundational studies established deep RL as a viable trader on this task [24–26], and later work improved robustness and sample efficiency through imitation of profitable behaviour [27, 28], multimodal or convolutional state encoders [29], and reward shaping for sparse-reward environments [30]. DeepScalper [1] adds a hindsight bonus and an auxiliary price-prediction task to improve generalisation in intraday trading and remains a strong representative of the single-agent design. The common thread is a single, monolithic policy: effective at its target frequency, but—as the technical chapters show—unable to absorb the horizon explosion of true high-frequency trading or the regime non-stationarity that erodes one policy over long deployments.

3.1.2 High-Frequency Trading

For high-frequency trading specifically, DRA [2] couples an LSTM [31] state encoder with PPO; CDQNRP [3] adds random perturbation to the target update to stabilise a convolutional DQN; and asynchronous deep double-duelling Q-learning has been applied to trading-signal execution at speed [32]. These methods concentrate on building one robust agent for short-term trading and, as a result, struggle to maintain performance as the market trend changes. The dominant alternative in practice remains rule-based technical analysis (MACD, IV), which is brittle to hyperparameters. EarnHFT and FineFT depart from the single-agent paradigm by training pools of specialists.

3.1.3 Portfolio Management and Asset Allocation

Beyond single-asset timing, a substantial line applies RL to portfolio management, where the action is a continuous allocation across many assets. Early frameworks cast allocation as a deterministic policy-gradient problem over price tensors [33, 34]; subsequent work added explicit risk–return balancing [35], ensemble strategies that switch among actor–critic learners [36], hierarchical decomposition of selection and

allocation [37, 38], multi-objective optimisation [39], and maskable representations that respect tradability constraints [40]. Margin Trader [41] extends the setting to leverage and short selling—the regime that Chapter 5 studies. Portfolio management is not the subject of this thesis, but it shares the central difficulty of non-stationarity and motivates much of the ensemble and hierarchical machinery reused here.

3.1.4 Order Execution

A complementary task is optimal order execution—acquiring or liquidating a large position while minimising market impact. Hierarchical RL has been used to split a parent order across time and price [15], while recent work stresses generalisation across instruments and regimes [5] and imitative learning with predictive state representations [42]. Execution is the task in which the structural assumption EarnHFT exploits—that at high frequency a single trader's orders do not move prices (Section 4.3.1)—is most visibly an approximation, and the literature here is correspondingly attentive to market impact.

3.1.5 Hierarchical and Routing Approaches

A growing line of work does not commit to one policy but routes among several. MetaTrader [16] learns a router that picks the most suitable strategy for the current market; mixture-of-experts designs train a set of diversified traders and combine them [43]; and classical online-learning routers such as Winnow [44] reweight predictors by their recent performance. EarnHFT [45] and MacroHFT [46] construct hierarchical MDPs for high-frequency trading and train the high-level router *by reinforcement learning on historical data*, with optimal-action-value supervision; the router is fitted offline and then frozen for deployment rather than updated online. What these routers nonetheless share is that selection is itself a learned or performance-driven objective—whether reweighted online from recent returns or trained offline against the trading reward—which, as this thesis argues, ties the aggregation mechanism to the very reward signal whose non-stationarity it is meant to absorb. FineFT and AlphaQD progressively decouple the aggregation mechanism from that learning target.

3.1.6 Formulaic Alpha Mining

Formulaic alpha mining searches for short symbolic expressions whose cross-sectional values predict returns. Hand-crafted catalogues such as Alpha101 [9] gave way to automated search. Genetic programming [47] and its hierarchical and warm-started variants [48, 49] evolve expression trees by mutation and crossover, controlling diversity through population dynamics and post-hoc correlation filters, while deep symbolic regression [50] trains a recurrent generator of expressions. The reinforcement-learning line begins with AlphaGen [51], which casts mining as a PPO problem whose terminal reward is the marginal contribution of a new alpha to a running pool. AlphaQCM [52] adds a distributional value head and a variance bonus; RiskMiner [53] pairs risk-seeking Monte-Carlo tree search with the same pool reward; Alpha^2 [54] searches logical formulas; AlphaForge [55] replaces the RL search with a generator–predictor surrogate and a dynamic re-combination stage; and AlphaSAGE [56] encodes expressions as typed graphs and trains a GFlowNet against a dense, multi-faceted reward. A parallel and very recent line replaces the search policy with a large language model [57–59]. As Chapter 6 details, every reinforcement-learning miner in this family routes pool information through the reward, inheriting the non-stationarity and absent generalisation bound that AlphaQD addresses.

3.1.7 Uncertainty and Non-Stationary Reinforcement Learning

The leverage setting of Chapter 5 foregrounds two kinds of uncertainty. *Aleatoric* uncertainty is the irreducible noise of the environment, which leverage amplifies; risk-aware quantile methods [60] target it directly. *Epistemic* uncertainty is the model's ignorance about states it has not seen, and is the proper target of OOD detection. Modelling slowly drifting environments as a dynamic-parametric or hidden-parameter MDP [61] provides the formal setting in which a short window can be treated as approximately stationary, an assumption FineFT exploits. Prior trading work largely overlooks epistemic uncertainty; FineFT makes competence-awareness explicit.

3.2 Open Challenges

This survey exposes three gaps that the technical chapters close.

- **Gap I (high-frequency efficiency and adaptivity).** Existing RL methods for high-frequency trading train a single agent and are defeated by both the million-step horizon and the regime non-stationarity of the market. A method is needed that is data-efficient at second scale and adaptive across regimes. This motivates EarnHFT (Chapter 4).
- **Gap II (risk and competence-awareness under leverage).** Methods designed for low-leverage markets ignore the amplified stochasticity and the catastrophic downside of leverage, and lack any awareness of their own competence boundary, leaving them exposed during black-swan events. This motivates FineFT (Chapter 5).
- **Gap III (reward stationarity and generalisation in alpha mining).** Pool-based RL alpha miners couple the reward to an evolving pool, training the policy against a moving target and offering no finite-sample generalisation bound. This motivates AlphaQD (Chapter 6).

A thread common to all three gaps is the placement of the aggregation mechanism. EarnHFT learns it, FineFT derives it heuristically from generative uncertainty estimates, and AlphaQD removes it from the reward entirely—a progression that the remainder of the thesis develops.

Part II

Reinforcement Learning for Trading

Chapter 4

EarnHFT: Hierarchical Reinforcement Learning for High-Frequency Trading

4.1 Introduction

High-frequency trading (HFT) uses computer algorithms to place and cancel orders at very short time scales, and accounts for a majority of volume in modern financial markets [62]. It is especially prevalent in the cryptocurrency market, whose round-the-clock trading removes overnight risk and whose dramatic price fluctuations create frequent short-lived opportunities. A profitable HFT strategy can outperform a low-frequency one, which is why it is pursued aggressively by quantitative traders.

Reinforcement learning has produced strong results on low-frequency tasks such as daily stock or futures trading [1, 2], but these methods rarely survive the move to HFT, for two reasons identified in Section 3.2 as Gap I.

- **The horizon is extremely long, so RL is data-inefficient.** A second-level agent must be evaluated over dozens of days, giving a horizon of roughly one million steps—for instance, $60 \times 60 \times 24 \times 12 = 1{,}036{,}800$ seconds over twelve days—against the few-thousand-step horizon of Atari benchmarks [4]. Such horizons demand far more data to converge [5], straining computation.

- **The market drifts, so a single agent cannot maintain performance.** Standard RL assumes the training and testing environments coincide. In cryptocurrency, the market trend changes substantially between them; an agent trained under one trend incurs large losses when the trend reverses.

This chapter presents EarnHFT, an **E**fficient hier**A**rchical **R**einforcement lear**N**ing method for **H**igh **F**requency **T**rading, whose three stages are summarised in Figure 4.1. The design follows the decompose–specialise–aggregate pattern of Section 1.4.

1. **Stage I (Q-teacher).** We exploit a structural property of execution—that at second scale a single trader's orders do not move prices, so future prices are independent of the policy—to compute, by dynamic programming, the optimal action value for every state. Used as a regulariser and a demonstrator, this "Q-teacher" accelerates and stabilises second-level training.
2. **Stage II (specialist pool).** We train hundreds of agents under different market-trend preferences, segment and label the market without hyperparameter tuning, and select a small pool of the most profitable specialists.
3. **Stage III (router).** We train a minute-level router that dynamically picks a second-level specialist from the pool to maintain performance as the market changes.

Across four cryptocurrency markets spanning bull, bear, and sideways conditions, EarnHFT outperforms six state-of-the-art baselines on six financial metrics, exceeding the runner-up by at least 30% in profitability.

4.2 Problem Formulation

The financial concepts—limit order book, OHLC, technical indicators, market order, position, and net value—are defined in Section 2.1.1. We restrict attention to a single asset with long-only positions, and the objective is to maximise net value $V_t = V_{ct} + H_t\, p_t^{b_1}$, where V_{ct} is cash and H_t the position valued at the best bid. The execution price of a market order walks the book, $E_t(M) = \sum_i (p_t^i \min(q_t^i, R_{i-1}))(1+\sigma)$, where R_{i-1} is the residual quantity after level i and σ the commission rate.

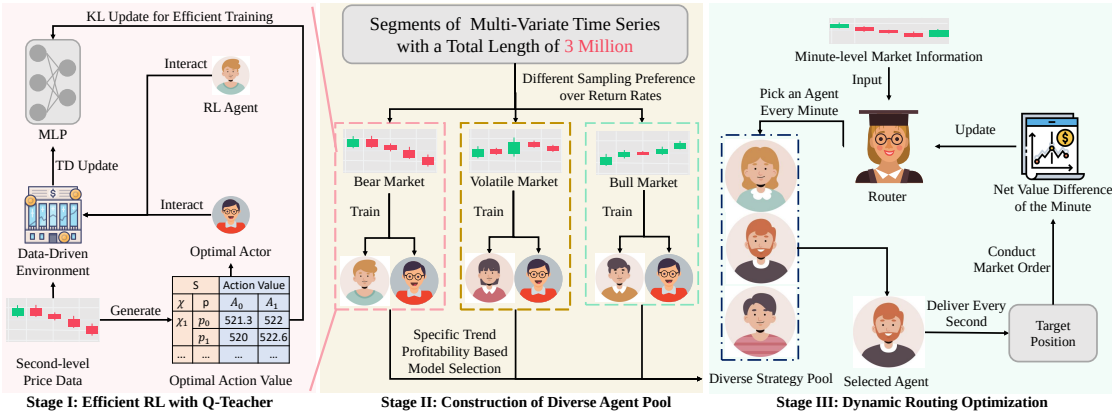


FIGURE 4.1: Overview of the three stages of EarnHFT. Stage I computes a Q-teacher to enhance the performance and training efficiency of second-level agents. Stage II trains diverse agents under various market trends and selects a small fraction by profitability to form an agent pool. Stage III trains a minute-level router that dynamically picks a second-level agent from the pool.

4.2.1 A Hierarchical Markov Decision Process

The two challenges of Section 4.1 have a common structural cause: micro-level information drifts, so no single agent profits under all trends, while macro-level information, an aggregation of the micro level, carries the slow trend signal. We therefore decompose HFT into a hierarchical MDP $(\text{MDP}_h, \text{MDP}_l)$, in which a low-level MDP at second scale models micro-level dynamics and execution, and a high-level MDP at minute scale models macro-level trend and strategy switching:

$$\text{MDP}_h = (S_h, A_h, P_h, R_h, \gamma_h, T_h), \qquad \text{MDP}_l = (S_l, A_l, P_l, R_l, \gamma_l, T_l).$$

Low level (second scale). The state S_{lt} comprises a latent representation y_{lt} of 54 micro-level technical indicators computed from a length-60 rolling window of second-level OHLC and LOB snapshots, together with the current position H_t. The action a_{lt} is a target position chosen from a finite pool $\{0, \frac{H}{|A|-1}, \ldots, H\}$; if it differs from the current position, a market order is executed immediately. The reward $r_{lt} = H_{t+1}p_{t+1}^{b_1} - (H_t p_t^{b_1} + E_t(H_{t+1} - H_t))$ is the per-second change in net value.

High level (minute scale). The state S_{ht} comprises a latent representation y_{ht} of 19 macro-level indicators from a length-60 rolling window of minute-level OHLC, together with H_t. The action a_{ht} selects an agent from the pre-trained pool A_h.

The reward $r_{hT} = \sum_{t=T}^{T+\tau} r_{lt}$ is the net-value change accrued by the selected low-level agent over one minute. In summary, every minute the high-level router picks a low-level agent, which then adjusts its position every second; the goal is to find a set of specialists and a router that jointly maximise total profit.

4.3 Method

4.3.1 Stage I: Efficient Reinforcement Learning with a Q-Teacher

A long trajectory normally inflates the cost of RL. In the low-level MDP, however, prices are not influenced by the policy, so future prices are known in hindsight on historical data. This lets us construct the optimal action value $Q^\star$ by dynamic programming [63] and use it to teach the agent, both as a value supervisor and as a demonstrator.

Optimal value supervisor. Because the position set is finite, the optimal action value can be computed backwards: from the last step, the value of each (position, action) pair is the immediate reward plus the best continuation value, as in Algorithm 1. We add to the DDQN loss a supervision term that is the Kullback–Leibler divergence between the agent's action values and the optimal action values at the same state:

$$L(\theta_i) = L_{td} + \alpha \, \mathrm{KL}(Q_t(\chi, p, \cdot\,; \theta_i) \,\|\, Q^\star(\chi, p, \cdot\,)) \,, \tag{4.1}$$

where $L_{td} = (r + \gamma \max Q_t(\chi', a, \cdot\,; \theta_i') - Q_t(\chi, p, a; \theta_i))^2$ is the DDQN temporal-difference error, χ the latent representation, p the position, and α a coefficient decaying over time. The supervision term lets the agent learn the relative advantage of all actions at a state without exploring them, accelerating training.

Optimal actor. Supervision alone is insufficient, because the optimal action value is conditioned on the current position: once the agent deviates from the optimal policy, the supervision target shifts, compounding the deviation. We therefore

Algorithm 1 Construction of the Optimal Action Value

Input: time series $\mathcal{D}$ of length N, commission rate δ, action space A
Output: table $Q^\star$ of optimal action values indexed by time, position, action
Initialise $Q^\star$ with shape $(N, |A|, |A|)$ and all elements 0.
for $t \leftarrow N-1$ **to** 1 **do**
    for $p \leftarrow 1$ **to** $|A|$ **do**
        for $a \leftarrow 1$ **to** $|A|$ **do**
            $Q^\star[t,p,a] \leftarrow \max_{a'} Q^\star[t+1,a,a'] + a\, p_{t+1}^{b_1} - (p\, p_t^{b_1} + E_t(p-a))$
        end for
    end for
end for
return $Q^\star$

additionally train on transitions generated by the optimal policy itself—the action with the highest optimal value—which supplies demonstration experience and prevents the agent from settling in a local trap. Algorithm 2 collects experience from both the ϵ-greedy policy and the optimal actor, then updates the network using both TD errors and the KL supervision.

Algorithm 2 Efficient Reinforcement Learning with a Q-Teacher

Input: time series $\mathcal{D}$ of length N, commission rate δ, action space A
Output: network parameters θ
Initialise replay buffer R, network Q_θ, target $Q_{\theta'}$; build $Q^\star$ via Algorithm 1; initialise environment.
for $t = 1$ **to** $N-1$ **do** ▷ exploration rollout
    Choose a_ϵ by the ϵ-greedy policy; store $(s, a_\epsilon, r, s', Q^\star)$ in R.
end for
for $t = 1$ **to** $N-1$ **do** ▷ optimal-actor rollout
    Choose $a_o = \arg\max_a Q^\star[t,p,a]$; store $(s, a_o, r, s', Q^\star)$ in R.
end for
Sample transitions; compute L by (4.1); descend on θ and set $\theta' = \tau\theta + (1-\tau)\theta'$.
return Q_θ

The Q-teacher does not compromise correctness. Treating the supervised update as an operator H on action values, one can show H is a contraction in the sup-norm whose fixed point is $Q^\star$, so the learned value still converges to the optimum. Following the convergence argument for Q-learning [64], it suffices to verify these two properties of H. The operator combines the usual Bellman backup with the

supervision term and is

$$Hq(x,a) = \lambda \sum_{y \in X} T(y \mid x,a)\big(r(x,a,y)+\gamma \max_b q(y,b)\big)+(1-\lambda)\Big(\tfrac{1}{|A|}\textstyle\sum_{a\in A} q(x,a)+Re(x,a)\Big), \tag{4.2}$$

where the supervision residual is $Re(x,a) = q^\star(x,a) - \frac{1}{|A|}\sum_{a\in A} q^\star(x,a)$ and $\lambda \in (0,1]$ weights the Bellman backup against the supervision term.

Proposition 4.1 ($Q^\star$ is a fixed point of H)**.** *The optimal action value $q^\star$ satisfies $Hq^\star = q^\star$.*

Proof. Substituting $q^\star$ into (4.2) and using the Bellman optimality equation $q^\star(x,a) = \sum_y T(y \mid x,a)(r(x,a,y) + \gamma \max_b q^\star(y,b))$ for the first term and the definition of Re for the second,

$$\begin{aligned} Hq^\star(x,a) &= \lambda\, q^\star(x,a) + (1-\lambda)\Big(\tfrac{1}{|A|}\textstyle\sum_a q^\star(x,a) + q^\star(x,a) - \tfrac{1}{|A|}\sum_a q^\star(x,a)\Big) \\ &= \lambda\, q^\star(x,a) + (1-\lambda)\, q^\star(x,a) = q^\star(x,a). \end{aligned}$$

□

Proposition 4.2 (Contraction in the sup-norm)**.** *For any action values q_1, q_2, $\|Hq_1 - Hq_2\|_\infty \le k\, \|q_1 - q_2\|_\infty$ with $k = 1-(1-\lambda)\gamma < 1$.*

Proof. The supervision residual $Re(x,a)$ depends only on $q^\star$ and not on the argument of H, so it cancels in the difference $Hq_1 - Hq_2$. Bounding the max operator by $|\max_b q_1(y,b) - \max_b q_2(y,b)| \le \max_b |q_1(y,b) - q_2(y,b)|$,

$$\begin{aligned} \|Hq_1 - Hq_2\|_\infty &\le \lambda\gamma\, \|q_1 - q_2\|_\infty + (1-\lambda)\, \|q_1 - q_2\|_\infty \\ &= \big(1-(1-\lambda)\gamma\big)\|q_1 - q_2\|_\infty. \end{aligned}$$

Since $\gamma \in (0,1]$ and $\lambda \in (0,1]$, the factor $k = 1-(1-\lambda)\gamma < 1$. □

Theorem 4.1 (Convergence of the supervised update)**.** *Under a finite MDP, the action value learned with the optimal-value-supervisor update of (4.1) converges to the optimal action value $Q^\star$.*

Proof. Propositions 4.1 and 4.2 show that H is a sup-norm contraction with unique fixed point $Q^\star$. By the stochastic-approximation argument of [64], the iterates therefore converge almost surely to $Q^\star$. □

4.3.2 Stage II: Construction of the Agent Pool

Because micro-level information changes rapidly, no single agent maintains performance over a long period; preliminary experiments confirm that training across conflicting trends is incompatible. We therefore decompose the market into trends and develop a specialist for each.

Generating diverse agents. Prior approaches obtain diversity as a by-product of random seeds or hyperparameter search [43], which is unstructured. We instead train agents with explicit *preferences* over market trends. The training series (over three million steps) is cut into chunks of length L, each a continuous trend; a preference parameter β assigns each chunk with buy-and-hold return rate r a sampling priority

$$f(x) = \begin{cases} \dfrac{e^{\beta r}}{\mathrm{pdf}(r)} & \text{if } Q_{\frac{\theta}{2}}(R) \le r \le Q_{1-\frac{\theta}{2}}(R), \\ e^{\beta r} & \text{otherwise,} \end{cases} \tag{4.3}$$

where $Q_{\frac{\theta}{2}}(R)$ is the $\frac{\theta}{2}$-quantile of return rates and pdf is a kernel density estimate,

$$\mathrm{pdf}(x) = \frac{1}{nh} \sum_{r \in R} K\left(\frac{x - r}{h}\right), \tag{4.4}$$

with bandwidth h searched around Silverman's rule [65] and K a standard normal kernel. Dividing by the density removes the influence of the dataset's own distribution, while $e^{\beta r}$ tilts sampling towards a preferred trend; clipping the extremes prevents a few violent chunks from poisoning the agent everywhere. Different β values yield agents specialised to different trends.

Market segmentation and labelling. To evaluate specialists per trend we need to label the market. Algorithm 3 denoises the series, splits it at extrema, merges adjacent segments whose dynamic-time-warping [66] and slope differences are small, and labels segments by slope quantiles. Unlike prior methods, it requires no per-dataset hyperparameter tuning.

Agent selection. Including all trained agents would inflate the router's action space, so we select a small pool. Each point of the validation set is labelled by

Algorithm 3 Market Segmentation and Labelling

1: **Input:** time series $\mathcal{D}$ of length N; risk threshold θ; number of labels M
2: **Output:** a trend label for every point of $\mathcal{D}$
3: $\mathcal{D}' \leftarrow$ denoise high-frequency noise of $\mathcal{D}$.
4: Divide $\mathcal{D}'$ at its extrema into segments S.
5: Merge adjacent segments in S whose DTW and slope differences are small, until S is stable.
6: Compute thresholds $H = Q_{1-\frac{\theta}{2}}(R)$, $L = Q_{\frac{\theta}{2}}(R)$ on segment slopes R.
7: Compute per-label slope bounds from the quantiles and thresholds; label each segment.
8: **return** the label of each segment.

Algorithm 3; agents are evaluated across trends and initial positions, and the most profitable agent under each (trend, initial-position) pair is retained, forming a two-dimensional pool indexed by (m, n) for m trends and n initial positions.

4.3.3 Stage III: Dynamic Routing

We train the router for the high-level MDP with DDQN [11]. Operating at minute scale shortens the trajectory by 98.33% relative to the second scale, and we further restrict the router, at each step, to agents whose initial position matches the current position, reducing the choice to m agents. We deliberately do *not* compute a Q-teacher for the router: the horizon is already short, and the high-level action is a strategy rather than a target position, which would make the optimal value expensive to compute. With the computation for self-exploration reduced and that for the teacher increased, plain DDQN is the more efficient choice at the high level.

4.4 Experimental Setup

Datasets. We evaluate on four cryptocurrencies covering mainstream and niche assets and bull, bear, and sideways conditions, summarised in Table 4.1. On each, the last nine days are used for testing, the preceding nine for validation, and the remainder for training. Low-level agents are trained on the training set; the validation set is segmented and labelled for model selection; the router is then trained on the training set and selected on the full validation set before testing.

TABLE 4.1: The four cryptocurrency datasets, with market dynamics, number of seconds, and chronological span. Dates are in YY/MM/DD format.

Dataset	**Dynamics**	**Seconds**	**From**	**To**
BTC/TUSD	Sideways	4,057,140	23/03/30	23/05/15
BTC/USDT	Sideways	3,884,400	22/09/01	22/10/15
ETH/USDT	Bear	3,970,800	22/05/01	22/06/15
GALA/USDT	Bull	3,970,740	22/07/01	22/08/15

Metrics. We report the six financial metrics defined in Section 2.1.3: TR (profit); ASR, ACR, ASoR (risk-adjusted profit); and AVOL, MDD (risk).

Baselines. We compare against six methods: two policy-based RL algorithms, PPO [12] and DRA [2]; two value-based RL algorithms, DQN [10] and CDQNRP [3]; and two rule-based methods, MACD [8] and IV [7]. Where official implementations exist we reuse their hyperparameters; otherwise we reimplement faithfully from the original papers.

Training. All experiments run on a single RTX 4090 GPU. The commission rate is 0 for BTC/TUSD and 0.02% for the others, following Binance. We train with $\beta \in \{-90, -10, 30, 100\}$ for 50 epochs each, producing 200 agents, optimised with Adam. The full set of experiments completes in about ten hours.

4.5 Results and Analysis

4.5.1 Comparison with Baselines

Table 4.2 reports the main comparison. EarnHFT achieves the highest profit (TR) on all four datasets and the highest risk-adjusted profit on three. Value-based methods (DQN, CDQNRP) perform acceptably only when the validation and test sets are similar under a stable trend. Policy-based methods (PPO, DRA) tend to collapse onto a do-nothing policy—returning a target equal to the current position to avoid commission—even at a learning rate of 10^{-7}, and fail badly in the bear market. Rule-based methods are highly sensitive to their take-profit and stop-loss

thresholds and earn only moderate profit in volatile markets. EarnHFT, by contrast, is a deliberately aggressive trader because the optimal-value supervisor and optimal actor convey profit-oriented experience; it consequently trails on some risk metrics while dominating on profit. Figure 4.2 shows a representative episode: EarnHFT opens and closes a position within thirty seconds, profiting from a movement that appears as a mere pullback at minute scale.

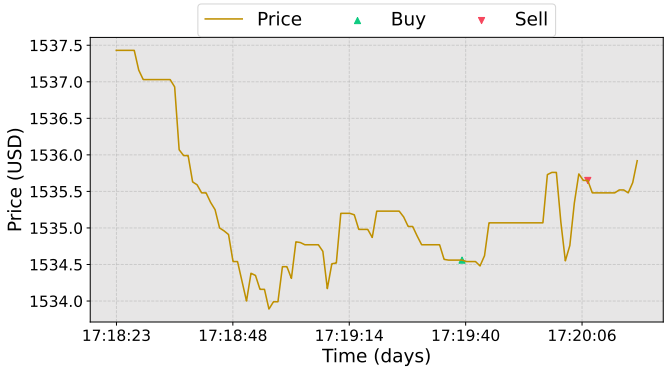


FIGURE 4.2: A representative EarnHFT trading episode on ETH. The agent opens and closes a position within roughly thirty seconds, capturing a movement that registers only as a pullback at the minute scale.

4.5.2 Effectiveness of the Hierarchical Framework

Figure 4.3 compares the router with each specialist in its pool. Bull and rally specialists tend to buy and hold and excel in the bull market (GALA); sideways specialists trade little and hold; pullback and bear specialists tend to close positions and excel in the bear market (ETH). The router combines these strengths and delivers the best profit on all four datasets. Figure 4.4 shows the router's selection distribution: high-volatility datasets (ETH, GALA) elicit a balanced distribution across trends because dynamics change frequently, whereas low-volatility datasets (BTCT, BTCU) concentrate on a few dynamics.

4.5.3 Effectiveness of the Optimal Action Value

Table 4.3 ablates the optimal value supervisor (OS) and optimal actor (OA) on ETH and GALA, measuring the steps to converge (CS), the converged reward sum (RS), and the average holding length (AHL). On GALA, adding OS to plain DDQN

TABLE 4.2: Performance comparison on four cryptocurrency markets against six baselines (two policy-based, two value-based, two rule-based RL/rule methods). Arrows indicate the preferred direction; pink marks the best entry per market and metric. TR, AVOL, and MDD are in percent.

Market	Model	TR↑	ASR↑	ACR↑	ASoR↑	AVOL↓	MDD↓
BTCU	DRA	−4.56	−4.28	−19.57	−4.65	42.25	9.24
	PPO	−3.61	−5.25	−22.74	−5.71	27.76	6.41
	CDQNRP	−2.83	−2.91	−14.85	−3.31	37.61	7.38
	DQN	−3.48	−12.37	−35.01	−11.86	11.57	4.09
	MACD	−6.07	−10.11	−25.17	−8.05	24.84	9.98
	IV	−2.99	−3.78	−14.24	−3.27	31.35	8.32
	EarnHFT	0.72	1.22	10.77	0.93	27.08	3.07
BTCT	DRA	−2.65	−4.82	−17.48	−4.77	21.18	5.84
	PPO	−0.60	−14.74	−35.80	−0.14	1.59	0.65
	CDQNRP	−0.60	−19.52	−37.88	−0.74	1.20	0.61
	DQN	0.47	4.21	27.94	1.14	4.38	0.66
	MACD	−4.02	−5.80	−24.21	−4.32	26.87	6.44
	IV	−12.01	−17.83	−38.99	−13.90	27.68	12.66
	EarnHFT	0.99	1.34	7.76	1.01	32.40	5.61
ETH	DRA	−33.37	−9.06	−32.23	−9.20	163.25	45.88
	PPO	−22.61	−10.11	−31.17	−10.39	96.12	31.17
	CDQNRP	−6.82	−24.41	−40.19	−3.11	11.46	6.96
	DQN	−11.02	−9.47	−32.81	−8.43	47.76	13.79
	MACD	−4.29	−1.78	−8.71	−1.19	79.64	16.35
	IV	−27.42	−12.27	−36.01	−9.00	99.67	33.96
	EarnHFT	4.52	2.92	14.30	1.78	67.92	13.89
GALA	DRA	10.56	4.77	41.63	4.44	92.43	10.60
	PPO	10.56	4.77	41.63	4.44	92.43	10.60
	CDQNRP	5.22	4.51	39.42	4.16	47.27	5.41
	DQN	2.94	3.55	32.02	2.66	34.08	3.78
	MACD	2.37	1.79	11.45	1.22	62.89	9.84
	IV	13.95	6.74	55.67	5.41	81.79	9.91
	EarnHFT	19.41	9.77	79.08	7.79	74.94	9.26

reduces the steps to converge to about 15% while raising the return; OA further raises the return at the cost of more steps. On ETH, a bull market, OS does not reduce CS but substantially raises the return. OS is the more effective component

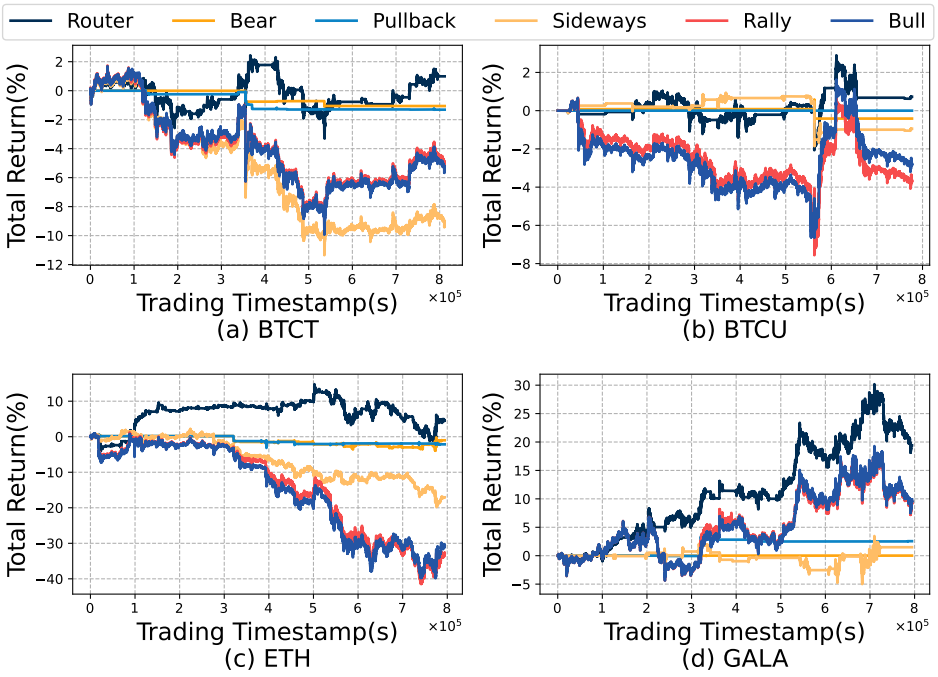


FIGURE 4.3: Comparison of the router against each specialist in its pool across the four datasets. Each specialist excels in the trend it was trained for; the router combines their strengths.

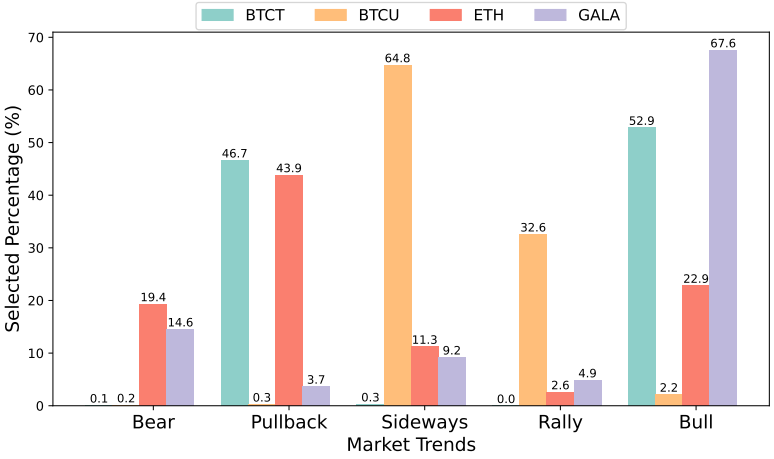


FIGURE 4.4: Distribution of the router's specialist selections across the four datasets. High-volatility markets elicit a more balanced distribution; low-volatility markets concentrate on fewer dynamics.

because it supplies information about all actions and adapts as the policy changes, whereas OA only provides demonstrations.

TABLE 4.3: Ablation of the optimal value supervisor (OS) and optimal actor (OA) on ETH and GALA. CS is steps to converge, RS the converged reward sum, AHL the average holding length.

OA	OS	GALA			ETH		
		CS↓	RS↑	AHL	CS↓	RS↑	AHL
✓	✓	78848	4.43	448	102400	12.32	81.3
✓		102400	0.24	38.7	102400	−1.40	4.15
	✓	4608	2.89	147	30720	4.87	35.8
		30720	−0.01	284	30720	−29.6	39.1

4.5.4 Trading Behaviour and Computational Cost

The trading behaviour of EarnHFT varies with the underlying market in a manner consistent with the unit cost of each asset and the stability of its trend. On the two Bitcoin datasets (BTCT and BTCU), the high unit price makes each round trip relatively expensive in dollar terms, and the agent trades less frequently and holds positions only when the price excursion is comfortably larger than the round-trip commission. On ETH, where the trend reverses sharply within the test window, the agent trades much more often to capture the short pullbacks that the minute-level router exposes. On GALA, which trends upward through most of the test period without major reversals, the agent holds positions for long stretches and lets the trend do the work. In aggregate, EarnHFT trades less frequently in stable bull markets and more frequently in volatile or bear markets—behaviour that matches what a discretionary high-frequency trader would do, but emerges from the hierarchy without any hand-coded regime detector.

Computational cost. All experiments run on a single NVIDIA RTX 4090 GPU. The full pipeline—building the Q-teacher table, training the 200 specialist agents under the four preference values $\beta \in \{-90, -10, 30, 100\}$, segmenting and labelling the validation set, selecting the pool, and training the minute-level DDQN router—completes in approximately ten hours for each market, dominated by stage II specialist training. At inference time the only on-policy work is one specialist DDQN forward pass per second and one router forward pass per minute, both of which fit comfortably inside the second-level decision budget on a commodity GPU.

4.6 Chapter Summary

This chapter presented EarnHFT, a three-stage hierarchical RL framework that makes high-frequency trading tractable for RL. It addresses the extremely long horizon with a dynamic-programming Q-teacher that, under the exogenous-price assumption of Section 4.3.1 and a fixed historical training window, preserves the optimal Bellman fixed point of the supervised update while accelerating learning empirically; and it addresses regime non-stationarity by training a pool of trend specialists and a minute-level router that switches among them. EarnHFT outperforms six strong baselines across four markets, exceeding the runner-up by at least 30% in profitability, and the ablations confirm the contribution of each component. In the language of the thesis, EarnHFT is the first instantiation of the decompose–specialise–aggregate pattern, and its aggregation mechanism—a DDQN router that selects a single specialist, trained by reinforcement learning against the trading reward—is the most tightly coupled of the three. The next chapter retains the pattern but confronts a setting, leveraged futures, where the router must additionally manage risk and the agent must become aware of its own competence boundary.

Chapter 5

FineFT: Risk-Aware Ensemble Reinforcement Learning for Futures Trading

5.1 Introduction

Futures are standardised contracts obligating the exchange of an asset at a predetermined date and price. They account for the majority of cryptocurrency trading volume [67] because they permit both long and short positions and offer high leverage, attracting risk-seeking traders and creating a deeply liquid market with capacity in the tens of trillions of dollars. Futures trading is therefore a crucial domain for quantitative methods, and the natural next task after the spot high-frequency trading of Chapter 4.

Reinforcement learning has succeeded across many trading tasks [45, 46], but those methods target low-leverage markets such as spot or stock and do not transfer to leveraged futures, for two reasons that constitute Gap II of Section 3.2 and are illustrated in Figure 5.1.

- **Leverage amplifies the stochasticity of the reward.** High leverage magnifies the inherent noise of financial markets [43]: even in nearly identical states, identical actions can lead to vastly different rewards and next states. This aleatoric uncertainty, visible in Figure 5.1(a) as wide variation in future

return for almost identical market-state values, undermines the convergence and stability of RL and reduces data efficiency.

- **Agents are unaware of their capability boundary.** Because RL overfits the training set, a policy becomes unreliable on market states unseen during training, exposing it to severe capital loss [68]. A black-swan event—such as the late-2022 shift triggered by aggressive interest-rate hikes, visible in Figure 5.1(b) as an abrupt jump of a state cluster—can liquidate an agent that lacks self-awareness of its competence. This is an *epistemic* uncertainty that low-leverage methods can afford to ignore but leveraged ones cannot.

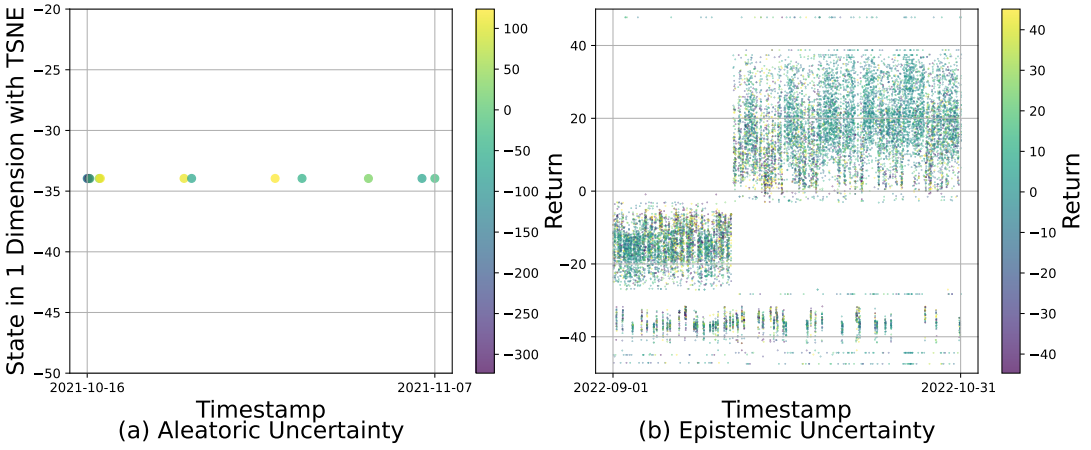


FIGURE 5.1: The two challenges RL faces in futures trading. The horizontal axis is time; the vertical axis is a one-dimensional t-SNE [69] embedding of the market state y_t; colour encodes the future five-minute return. *(a)* Aleatoric uncertainty: nearly identical state values yield widely varying future returns. *(b)* Distribution shift: a state cluster abruptly relocates during a macro event.

This chapter presents FineFT, an e**F**ficient and r**I**sk-aware e**N**semble r**E**inforcement learning method for **F**utures **T**rading, with three stages summarised in Figure 5.2. It follows the same decompose–specialise–aggregate pattern as EarnHFT, but the aggregation mechanism is now risk-aware and the agent is made competence-aware.

1. **Stage I (selective-update ensemble).** We initialise N deep Q-networks, pre-train them on demonstrations, and then update them *selectively*: for each transition we update only the learner with the smallest temporal-difference error and its neighbours. This trains diverse specialists efficiently and segments the market automatically.

2. **Stage II (filter and boundary).** We filter the ensemble by profitability under each market dynamic to shrink the pool, and train a variational autoencoder per dynamic to capture each specialist's capability boundary and detect out-of-distribution states.

3. **Stage III (risk-aware routing).** Guided by VAE losses on recent states, we route to the best in-distribution specialist for profit, and fall back to a conservative policy on out-of-distribution states to control risk.

On crypto futures with 5× leverage, FineFT outperforms twelve state-of-the-art baselines on six metrics, reducing risk by more than 40% relative to the runner-up while remaining the most profitable learned method.

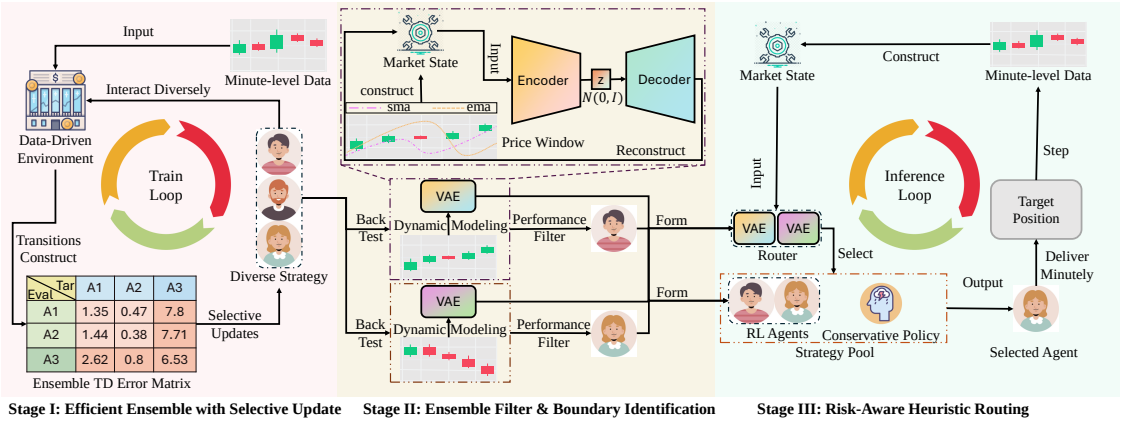


FIGURE 5.2: Overview of the three stages of FineFT. Stage I computes ensemble TD errors for selective updates. Stage II filters the ensemble by performance across dynamics and trains a VAE per dynamic to identify capability boundaries. Stage III routes among the ensemble and a conservative policy for stable performance and risk reduction.

5.2 Problem Formulation

Beyond the concepts of Section 2.1.1, futures trading introduces *leverage* L_t, the ratio of position value to required margin; the *mark price* M_t; a *market order loss* O_t that captures slippage and commission; a *funding fee* $F_{ft} = F_{rt}H_t$ paid to hold a leveraged position; and the *margin balance* V_t, the sum of cash and position value net of realised costs. The objective is to maximise the margin balance via leveraged market orders on a single asset at minute scale.

5.2.1 A Dynamic Parametric MDP

To make the non-stationarity explicit, we model futures trading as a dynamic parametric MDP (DP-MDP) [61], the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma, T, Z, P_z)$ that augments the MDP of Section 2.2.1 with a *dynamic* space Z and its transition P_z. The dynamic z governs the transition $P : \mathcal{S} \times Z \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ but is not revealed to the agent, and it evolves much more slowly than the state. Crucially, this means that *over a short horizon the experienced transitions behave as those of a stationary MDP*—the assumption that legitimises training a specialist per dynamic.

The goal is to learn a set of specialist policies $\Pi_A = (\pi_{\theta_1}, \ldots, \pi_{\theta_m})$ and a router $\pi_{\theta_r} : \mathcal{S} \to Z$ that compose an optimal strategy $\pi^\star = \arg\max_{A,\theta_r} \mathbb{E}_{\Pi_A,\pi_{\theta_r}}[\sum_{t=0}^{T} \gamma^t r_t]$. The state S_t consists of around 300 technical indicators y_t and the position H_t; the reward is the margin-balance change

$$r_t = V_{t+1} - V_t = H_t(M_{t+1} - M_t) - O_t, \tag{5.1}$$

combining valuation change and order loss; and the action is a target (position, leverage) pair drawn from a finite pool, executed instantly by a market order.

5.3 Method

5.3.1 Stage I: Efficient Ensemble with Selective Update

A mixture of experts performs well on trading tasks but is expensive to construct, because every specialist is trained independently [43, 45]. FineFT instead trains the ensemble jointly: all learners share a pre-training phase that installs common knowledge, after which each transition updates only the best-fitting learner and its neighbours. The aim is an ensemble in which each learner masters one dynamic z of the DP-MDP.

Each of the N learners is a multilayer perceptron with identical architecture but a distinct random seed, assigned an index $1, \ldots, N$.

Pre-training with demonstrations. As new traders are trained together before specialising, we first train all learners on the same demonstration transitions. Following the principle of Chapter 4, we build a *constrained* optimal action value (Algorithm 4) by adding a liquidation mask to the dynamic-programming construction, and use it both as an optimal actor (to generate demonstrations) and an optimal value supervisor. Three auxiliary policies—emptying, maximally long, maximally short—add diverse transitions. Each learner is updated with a Huber TD error [70], robust to the outliers leverage produces, plus the KL supervision:

$$L(\theta_i) = L_{td_i} + \alpha\,\mathrm{KL}(Q(s,\,\cdot\,;\theta_i)\,\|\,Q^\star(s,\,\cdot\,))\,, \quad L_{td_i} = \mathcal{H}(r + \gamma \max Q(s,\,\cdot\,;\theta_i') - Q(s,a;\theta_i))\,, \tag{5.2}$$

where $\mathcal{H}$ is the Huber loss, quadratic for small errors and linear for large ones. The total pre-training loss sums (5.2) over all learners.

Algorithm 4 Constrained Optimal Action Value

Input: LOB series B, mark-price series M of length N, action space A, mask penalty P, capital C
Output: table $Q^\star$ of optimal action values
Initialise $Q^\star$ with shape $(N, |A|, |A|)$ and all elements 0.
for $t \leftarrow N-1$ **to** 1 **do**
  for $p, a \leftarrow 1$ **to** $|A|$ **do**
    $Q^\star[t,p,a] \leftarrow \max_{a'} Q^\star[t+1,a,a'] + r_t$ ▷ r_t via (5.1)
    if $V_t \le C$ **then**
      $Q^\star[t,p,a] \leftarrow Q^\star[t,p,a] - P$ ▷ liquidation mask
    end if
  end for
end for
return $Q^\star$

Selective update with neighbours. Although dynamics share basic principles, their most profitable strategies conflict; diversity therefore matters. Prior work obtains diversity by training learners independently with preference sampling [45], at high cost. FineFT instead shares transitions selectively. For each transition we identify the learner with the smallest TD error—that is, the most accurate Q-value—and assign larger update weight to that learner and to its index-neighbours. Learners that fit poorly on this transition are left untouched, because their competence lies in a different dynamic that the transition would only corrupt. This creates a positive feedback loop: as a learner specialises in a dynamic, its TD

error there falls, so future transitions from that dynamic are increasingly routed to it, which reinforces the specialisation.

Concretely, for each transition we form an ensemble TD-error matrix L with entries

$$L_{ij} = \mathcal{H}\big(r + \gamma \max Q(s', \cdot; \theta'_j) - Q(s, a; \theta_i)\big)\,, \tag{5.3}$$

the cross-learner Huber TD error (L_{ii} is learner i's own error). Algorithm 5 converts L into a weight matrix $\mathcal{W}$: it locates the best learner $i^\star = \arg\min_i L_{ii}$, expands a neighbourhood of radius m, and assigns diagonal weights that decay with distance from $i^\star$ and off-diagonal weights that decay with index difference. The total selective-update loss for a transition is

$$L = \sum_{i=1}^{N}\sum_{j=1}^{N} \mathcal{W}_{ij} L_{ij} + \mathbf{d}_{\mathcal{W}}\, O_{\mathrm{KL}}^{\top}, \tag{5.4}$$

where $\mathbf{d}_{\mathcal{W}}$ is the diagonal of $\mathcal{W}$ and O_{KL} collects the per-learner KL supervision. The cross-learner terms share experience among learners of similar dynamics while preventing experience poisoning across dissimilar ones.

Convergence of the selective update. The selective-update mechanism preserves convergence of each learner on the dynamic it specialises in. The argument has two parts. First, the pre-training loss (5.2) updates every learner with the same optimal-value supervisor used by EarnHFT, now constrained by the liquidation mask of Algorithm 4. The per-learner update operator is exactly the operator H of (4.2) in Chapter 4, so by Theorem 4.1 each learner's action value converges to the constrained optimal action value on the demonstration distribution. Second, consider the selective-update phase restricted to transitions from a single dynamic z. By the DP-MDP assumption of Section 5.2.1, those transitions are generated by a stationary MDP M_z. The weight matrix of Algorithm 5 assigns a non-negative weight to the best-fitting learner $i^\star$ and its neighbours and zero elsewhere; the diagonal weight of $i^\star$ is one. The update of learner $i^\star$ on transitions of dynamic z is therefore a standard Huber-TD update under M_z with a non-negative supervision term, whose update operator is a sup-norm contraction by the same argument. The positive-feedback property—specialising in z lowers learner $i^\star$'s TD error on z, which raises its selection weight on subsequent z-transitions—ensures that, once a learner becomes the best fit for z, it continues to receive the z-transitions that

drive its convergence on M_z. The cross-learner weights of neighbouring learners are bounded by the diagonal weights and decay with index distance, so they perturb the update by a contraction-preserving convex combination and do not break the contraction. Hence each specialist converges to the optimal action value of the dynamic it is selected for.

Algorithm 5 Construction of the Weight Matrix

Input: ensemble TD-error matrix $\mathcal{L}$, neighbour count m
Output: weight matrix $\mathcal{W}$
Initialise $\mathcal{W}$ with shape (N, N) and all elements 0.
$i^\star \leftarrow \arg\min_i L_{ii}$; $\quad i_{\min} \leftarrow \max(i^\star - m, 1)$; $\quad i_{\max} \leftarrow \min(i^\star + m, N)$
for $i = i_{\min}$ **to** $i_{\max}$ **do**
    $\mathcal{W}_{ii} \leftarrow 1 - \dfrac{|i - i^\star|}{i_{\max} - i_{\min}}$ ▷ diagonal decay
end for
for $i, j \leftarrow 1$ **to** N **do**
    if $i \neq j$ **then**
        $\mathcal{W}_{ij} \leftarrow \min(\mathcal{W}_{ii}, \mathcal{W}_{jj}) \left(1 - \dfrac{|i - j|}{i_{\max} - i_{\min}}\right)$ ▷ off-diagonal decay
    end if
end for
return $\mathcal{W}$

5.3.2 Stage II: Ensemble Filter and Boundary Identification

Knowing which dynamic each learner suits is necessary but not sufficient for safe routing; we must also know the boundary within which each learner is competent.

Filtering by profitability. We divide the market into dynamics by slope, following Chapter 4: the series is low-pass filtered, segmented at local extrema, and adjacent chunks with similar slope are merged until stable, yielding m dynamics. We back-test each learner across initial positions and retain, for each dynamic, the learner with the highest average profitability, producing a small, diverse, profitable pool.

Capability boundary. We define the *capability boundary* of the specialist retained for dynamic i as the set of market-state representations on which that

specialist was effectively trained—operationally, the support of the per-dynamic state distribution as estimated by a generative model. A state inside the boundary is one the model can reconstruct well; a state outside it is, by the same model, out of distribution. Reporting the boundary explicitly converts the agent's own competence from an implicit assumption of the training set into an estimable quantity at deployment, and it is the object the variational autoencoders of the next paragraph model. The boundary is established once on the training and validation windows and is held fixed at test time: the risk threshold of stage III is tuned on the validation window—never on the test window—so the deployment policy makes no use of test-time hindsight.

Boundary identification with VAEs. To capture each learner's capability boundary we train a variational autoencoder [19] on the market-state representations of each dynamic. A VAE encodes its input into a regularised latent distribution and decodes a reconstruction; because it gives a probabilistic measure of how well a new input matches the training distribution, it is a principled out-of-distribution detector. For m dynamics we train m VAEs, each on the states Y_i of its dynamic, minimising

$$L^i(y \in Y_i) = N_{LL}\big(\mu_d, \tfrac{1}{2}\log\sigma_d^2, y\big) + \mathrm{KLD}\big(\mu, \log\sigma^2\big)\,, \tag{5.5}$$

the reconstruction negative log-likelihood plus the normal-distribution regulariser. After training we record, for each VAE M^{v_i}, the negative loss on every state of its dynamic as reference scores R^i, used at deployment.

5.3.3 Stage III: Risk-Aware Heuristic Routing

Conservative policy. For states never seen during training or validation we design a conservative policy: hold the position if the single-trade drawdown stays under 5%, otherwise close it. This policy only closes, never opens, reducing position churn and ensuring losses are cut once a stop-loss triggers on unfamiliar states.

Routing with VAE losses. Training a high-level RL router is data-inefficient [45, 46], so we route heuristically. Given a window of recent states $Y_T^u = (y_{T-u}, \ldots, y_T)$,

each state's score under dynamic i is its empirical-CDF position among the reference scores R^i, smoothed by an exponential moving average:

$$R_t^i = F_{R^i}\left(-L^i(y_t)\right), \quad Re_{T_t}^{iu} = \gamma\, Re_{T_{t-1}}^{iu} + R_t^{iu}, \quad Re_T^{iu} = Re_{T_T}^{iu}. \tag{5.6}$$

We select the dynamic $i = \arg\max_i Re_T^{iu}$ and compare its score with a risk threshold τ: if it falls below τ, the recent states are unfamiliar and we choose the conservative policy; otherwise we deploy the corresponding specialist (Algorithm 6). This is the second instantiation of the thesis's aggregation mechanism—no longer a learned router, but a heuristic driven by generative uncertainty estimates, and hence partly decoupled from the policy learning.

Algorithm 6 Risk-Aware Heuristic Routing

Input: specialists $\Pi = (\pi^1, \ldots, \pi^m)$, VAEs M^v, reference scores R, state window Y_T^u, decay γ, threshold τ
Output: current strategy π
for dynamic $i = 1$ **to** m **do**
    Obtain state scores R_t^i from M^{v_i}, Y_T^u, and R^i.
    Compute the EMA score Re_T^{iu} by (5.6).
end for
$i \leftarrow \arg\max_i Re_T^{iu}$
if $Re_T^{iu} \le \tau$ **then**
    return conservative policy π^c ▷ out-of-distribution
else
    return specialist π^i
end if

5.4 Experimental Setup

Datasets. We evaluate on four mainstream cryptocurrencies over more than two years, covering bull, bear, and volatile conditions (Table 5.1). On each, the last six months are used for testing, the preceding six for validation, and the first year for training. The ensemble is trained on the training set; the validation set is segmented and labelled to filter the ensemble and train the VAEs, and to tune the routing hyperparameters (threshold τ, decay γ, window length).

Baselines. We compare against twelve methods: two policy-based (PPO [12], DRA [2]); two value-based (DQN [10], CRP [3]); two rule-based (MACD [8], IV [7]);

TABLE 5.1: The four cryptocurrency futures datasets, with the dynamic of the test period, number of steps, and chronological span. Dates are in YY/MM/DD format.

Dataset	Test dynamic	Size	From	To
BNB/USDT	Bear	210,241	22/01/01	24/01/01
BTC/USDT	Bull	210,241	22/01/01	24/01/01
DOT/USDT	Sideways	210,241	22/01/01	24/01/01
ETH/USDT	Sideways	210,241	22/01/01	24/01/01

two ensemble (EDQN [71], SUNRISE [14]); one distributional (RAQR [13]); and three hierarchical (WINOW [44], EHFT [45], MHFT [46]). The same hyperparameters used by FineFT are applied to the baselines where applicable.

Training. Experiments run on four RTX 4090 GPUs, completing in about six hours. The transaction cost is 0.02%, following Binance, with a leverage factor of 5. We report the six metrics of Section 2.1.3.

5.5 Results and Analysis

5.5.1 Comparison with Baselines

Table 5.2 reports the main comparison. FineFT attains the highest risk-adjusted profit (ASR, ACR, ASoR) on three of the four markets and the strongest risk control (lowest MDD) among methods that maintain meaningful market exposure. Several patterns are visible. Value-based methods (DQN, CRP) perform well only when the trend is stable and the validation–test gap is small. CRP outperforms DQN, confirming the value of training stability under high stochasticity. Policy-based methods (DRA, PPO) generally beat DQN but fail to surpass CRP, which suggests premature convergence to suboptimal policies in non-stationary environments. The small gap between DRA and PPO also indicates that an LSTM encoder helps little under such noise. Rule-based methods depend heavily on the validation–test similarity that fixes their hyperparameters. Ensemble (EDQN, SUNRISE) and distributional (RAQR) methods reduce variance, and so reduce risk. RAQR does so, however, by trading almost nothing: its near-zero volatility on DOT and

ETH reflects negligible participation rather than skilful control. Hierarchical methods (WINOW, EHFT, MHFT) achieve markedly higher profit and depend less on validation–test similarity, but exhibit large drawdowns precisely because they lack any awareness of their capability boundary. FineFT retains hierarchical-level profitability on familiar states while turning conservative on unfamiliar ones, thereby avoiding the large losses the other hierarchical methods incur.

5.5.2 Effectiveness of the Selective-Update Ensemble

Table 5.3 compares FineFT's pre-trained selective update (FP) against three alternatives on five dynamics: EHFT-style prioritised episode selection (EP), random dataset selection (ER), and FineFT without pre-training (FwoP). Convergence steps (CS) measure efficiency and reward sum (RS) the converged performance. FP converges in the fewest steps while achieving the strongest converged performance, confirming that sharing transitions selectively—rather than training each learner independently—improves both efficiency and the performance ceiling.

5.5.3 Effectiveness of Risk-Aware Routing and Case Studies

The risk-aware router is the source of FineFT's risk reduction. As the test period drifts further from the training and validation windows, the proportion of conservative-policy selections rises, and FineFT outperforms both its no-risk-threshold variant and any single specialist. Two case studies make the mechanism concrete. First, Figure 5.3 visualises the relationship between the updated learner index and the market dynamic during selective update: distinct learners specialise in distinct dynamics—some in sudden surges, others in sharp drops, others in minor fluctuations—confirming that the ensemble does not learn randomly but partitions the market. Second, Figure 5.4 shows the VAE proactively closing a BNB position upon encountering unfamiliar states, despite no closing signal from the selected specialist, avoiding the loss from a subsequent decline that coincided with an on-chain upgrade event.

TABLE 5.2: Performance comparison on four crypto futures markets against twelve baselines (four plain, two ensemble, one distributional, three hierarchical, two rule-based). Arrows indicate the preferred direction; pink marks the best entry per market and metric. TR, AVOL, and MDD are in percent. RAQR's near-zero risk on DOT and ETH reflects negligible market participation.

Market	Model	TR↑	ASR↑	ACR↑	ASoR↑	AVOL↓	MDD↓
	DRA	−64.79	−1.51	−2.58	−13.43	6.53	72.88
	PPO	−59.58	−1.64	−2.59	−14.12	5.55	67.37
	CRP	−64.79	−1.43	−2.59	−12.98	6.92	72.88
	DQN	−55.85	−1.21	−2.14	−13.72	6.09	66.09
	MACD	−82.85	−3.04	−4.12	−22.67	6.28	88.70
	IV	−71.00	−1.59	−2.73	−14.31	6.59	73.21
BNB	EDQN	−64.79	−1.51	−2.58	−13.43	6.53	72.88
	SUNRISE	−66.75	−1.74	−2.80	−14.45	6.27	74.32
	RAQR	−68.88	−1.84	−3.10	−14.22	6.47	73.27
	WINOW	86.65	2.01	5.99	31.71	5.46	34.93
	EHFT	−69.32	−2.08	−3.26	−17.31	6.05	73.61
	MHFT	−50.42	−0.81	−1.48	−9.10	6.47	67.95
	FineFT	83.70	2.55	11.19	101.17	3.57	15.49
	DRA	21.93	0.94	2.12	4.90	6.90	58.28
	PPO	19.68	0.90	2.01	4.39	6.83	58.30
	CRP	19.61	0.85	1.79	4.62	6.44	58.30
	DQN	−82.13	−12.03	−4.94	−51.81	1.77	82.37
	MACD	−26.28	−0.17	−0.31	−9.44	5.91	60.17
	IV	−17.26	0.01	0.02	−4.26	5.80	63.25
BTC	EDQN	25.11	0.98	2.15	5.88	6.80	59.28
	SUNRISE	−7.91	0.37	0.76	−2.06	6.87	63.36
	RAQR	2.84	0.57	1.10	0.71	5.19	51.19
	WINOW	−34.03	−0.23	−0.56	−8.62	7.09	55.30
	EHFT	33.44	1.42	2.73	10.78	4.29	42.61
	MHFT	−25.85	−0.32	−0.57	−7.27	4.46	47.97
	FineFT	76.90	2.13	7.44	39.51	4.29	23.51
	DRA	−65.15	−1.63	−2.63	−15.21	6.20	73.36
	PPO	−65.65	−1.63	−2.64	−15.18	6.27	73.80
	CRP	−73.13	−4.12	−4.09	−27.08	3.93	75.58
	DQN	−64.89	−1.35	−2.40	−13.66	6.84	73.42
	MACD	26.07	1.10	2.72	6.24	7.74	60.01
	IV	−66.24	−1.67	−2.70	−15.32	6.26	73.80
DOT	EDQN	−65.65	−1.63	−2.64	−15.18	6.27	73.80
	SUNRISE	−59.18	−1.02	−1.84	−12.51	6.94	73.60
	RAQR	−4.45	−4.47	−1.76	−45.61	0.10	4.72
	WINOW	−81.86	−4.17	−4.63	−26.72	4.85	83.35
	EHFT	89.00	1.95	4.09	19.78	6.28	57.18
	MHFT	220.44	3.48	10.47	65.02	5.10	32.44
	FineFT	18.39	1.08	2.50	14.46	2.64	21.87
	DRA	−46.56	−0.77	−1.44	−10.86	6.18	63.20
	PPO	−46.08	−0.74	−1.39	−10.68	6.23	63.01
	CRP	−46.06	−0.95	−1.69	−11.36	5.88	62.96
	DQN	−73.15	−3.94	−3.88	−27.31	3.89	75.48
	MACD	−73.93	−2.78	−3.49	−21.52	5.30	80.85
	IV	−50.53	−1.04	−1.82	−12.29	5.87	63.87
ETH	EDQN	−46.00	−0.73	−1.39	−10.65	6.23	62.96
	SUNRISE	−81.22	−4.45	−4.57	−23.85	4.48	83.36
	RAQR	−5.94	−4.10	−2.65	−13.75	0.21	6.08
	WINOW	26.36	1.06	3.55	6.98	6.59	37.55
	EHFT	48.72	1.49	3.33	10.66	5.65	48.33
	MHFT	−75.16	−2.21	−3.26	−15.37	6.32	81.79
	FineFT	39.05	1.74	5.78	14.45	2.96	16.96

TABLE 5.3: Convergence and performance of the selective-update ensemble against three training schemes on five dynamics: EHFT prioritised episode selection (EP), random selection (ER), FineFT with pre-training (FP), and without pre-training (FwoP). CS is convergence steps; RS is reward sum. Pink and green mark the best and second-best.

Method	CS↓	Dyn	RS↑	Method	CS↓	Dyn	RS↑
		1	2.86			1	2.86
		2	2.14			2	2.15
EP	71712	3	0.50	ER	112320	3	0.48
		4	4.98			4	4.98
		5	9.14			5	9.14
		1	45.29			1	37.34
		2	18.21			2	16.18
FP	66528	3	4.77	FwoP	79488	3	6.39
		4	24.48			4	25.53
		5	40.23			5	46.17

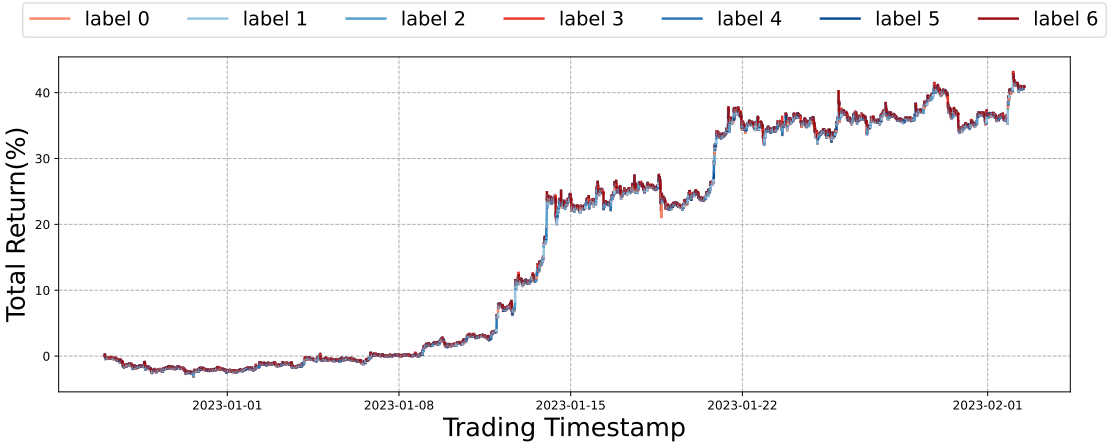


FIGURE 5.3: BTC return curve coloured by the index of the learner updated under selective update. The horizontal axis is time and the vertical axis cumulative return; distinct learners (colours) specialise in distinct market dynamics rather than learning interchangeably.

5.5.4 Parameter Sensitivity and Computational Cost

The two hyperparameters that most directly shape FineFT's profit–risk trade-off are the size of the specialist pool and the drawdown threshold of the conservative policy. Table 5.4 reports a small grid over the number of agents $\{3, 5, 7\}$ at the canonical drawdown threshold (5%) and over the drawdown threshold $\{1\%, 5\%, 10\%\}$ at the canonical pool size (5), measured on the BTC futures test window.

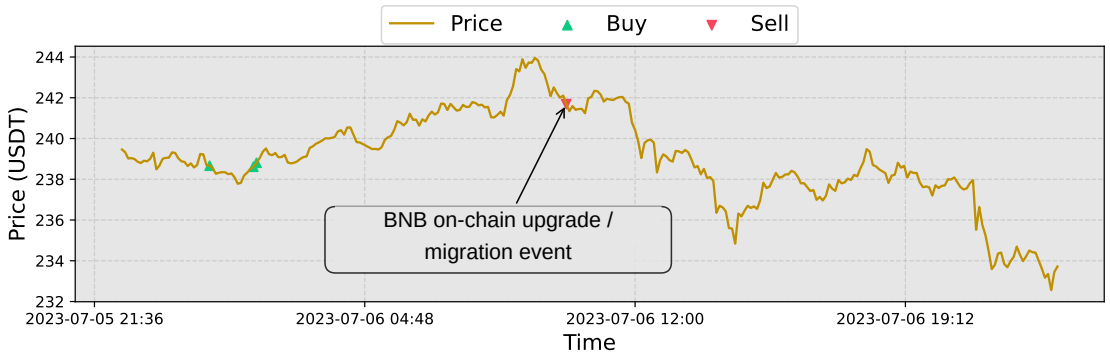


FIGURE 5.4: VAE-driven risk control on BNB. The horizontal axis is time and the vertical axis price; red and green markers are FineFT's sell and buy signals; the callout marks a BNB on-chain upgrade/migration event. The position is closed proactively when the rolling-window VAE loss is high, ahead of the subsequent decline.

TABLE 5.4: Sensitivity of FineFT to the number of specialists and the drawdown threshold (DDT) of the conservative policy. The canonical configuration (5 agents, 5% DDT) is the one reported in Table 5.2; the other rows vary one axis at a time. TR is total return, SR is Sharpe ratio, MDD is maximum drawdown.

#Agents	DDT	TR(%)↑	SR↑	MDD(%)↓
3	5%	52.74	1.25	39.10
5	5%	76.90	2.13	23.51
7	5%	59.63	1.46	37.50
5	1%	53.82	1.38	22.96
5	5%	76.90	2.13	23.51
5	10%	71.32	1.64	41.85

The pool size shows the expected diversification-versus-specialisation trade-off. Three agents leave too few specialists for the market dynamics observed and underperform the canonical five; seven agents over-fragment the training data per learner and the gain in specialisation does not compensate. Five sits at the sweet spot. The drawdown threshold is, by design, a risk-preference dial: a tight 1% cap delivers the lowest maximum drawdown but cuts return, while a loose 10% cap recovers some return at the cost of much higher drawdown. The default 5% converts the bulk of the risk reduction into retained return. Both axes vary smoothly and predictably; FineFT is not perched on a knife-edge configuration.

Computational cost. Training the full FineFT pipeline—selective-update ensemble pre-training, per-dynamic VAE fitting, and validation-window hyperparameter tuning—takes approximately six hours on a server with four NVIDIA RTX 4090 GPUs and an AMD Ryzen Threadripper PRO 5995WX CPU. At inference time the cost is dominated by the joint evaluation of one DQN specialist together with the five VAEs that gate it. With PyTorch's `vmap`, joint inference completes in approximately 0.78 ms on a single RTX 4090, only marginally above the 0.28 ms of the bare DQN; both are several orders of magnitude below the minute-scale decision frequency, so the VAE machinery does not constrain the deployment frequency in practice.

5.5.5 Generalisation to Other Asset Classes

The main evaluation is confined to cryptocurrency futures, the setting for which FineFT was designed. To test whether the method transfers beyond crypto, we apply it unchanged to two traditional financial futures with markedly different microstructure: corn futures, a commodity contract, and E-mini S&P 500 (ES) futures, an equity-index contract. Both are sampled at daily frequency. For each market we train on 2001–2013, validate on the following roughly five years, and test on the final roughly five years (about 3,000 training and 1,200 test days per market). These series are several orders of magnitude shorter than the crypto datasets—thousands of steps against nearly one million—so the experiment carries less statistical power and is reported as a generalisation check rather than as a primary result.

Table 5.5 compares FineFT against a 5× leveraged buy-and-hold benchmark and a plain DQN on total return, Sharpe ratio, and maximum drawdown. On both markets FineFT attains the highest total return and Sharpe ratio and the lowest maximum drawdown: it cuts the maximum drawdown of leveraged buy-and-hold to roughly a third while improving total return, and it improves on every metric over the DQN. The capability-boundary machinery therefore continues to convert risk control into retained return on asset classes and data frequencies it was not tuned for, indicating that the framework is not specific to cryptocurrency.

TABLE 5.5: Generalisation of FineFT to a commodity future (corn) and an equity-index future (E-mini S&P 500, ES), against a 5× leveraged buy-and-hold benchmark and a plain DQN, on the daily test windows (about 2018–2022). TR is total return, SR the Sharpe ratio, MDD the maximum drawdown; TR and MDD are in percent. Pink marks the best entry per market and metric.

Market	Model	TR(%)↑	SR↑	MDD(%)↓
Corn	Buy&Hold (5×)	440.71	1.04	91.93
	DQN	503.17	1.27	47.37
	FineFT	730.86	1.74	30.47
ES	Buy&Hold (5×)	288.96	0.88	90.64
	DQN	316.30	1.18	35.31
	FineFT	374.70	1.27	28.74

5.6 Chapter Summary

This chapter presented FineFT, a three-stage risk-aware ensemble RL framework for leveraged futures. A selective-update mechanism, driven by an ensemble TD-error matrix, trains diverse specialists efficiently and segments the market automatically. A per-dynamic VAE captures each specialist's capability boundary. At deployment, the router switches between specialists and a conservative fallback by out-of-distribution detection. FineFT outperforms twelve baselines, reducing maximum drawdown by more than 40% relative to the runner-up while remaining the most profitable learned method. Relative to EarnHFT, FineFT advances the thesis on two fronts. It makes the agent aware of its competence, which addresses the epistemic uncertainty that leverage renders fatal. It also replaces the learned router with a heuristic one driven by generative uncertainty estimates, which begins to decouple aggregation from policy learning. The next chapter completes this trajectory: it moves from trading to signal generation and removes the aggregation mechanism from the learning reward altogether.

Part III

Reinforcement Learning for Signal Generation

Chapter 6

AlphaQD: Quality-Diversity Reinforcement Learning for Formulaic Alpha Mining

6.1 Introduction

The two preceding chapters made trading *decisions*: an agent observed the market and chose how to adjust its position. This chapter changes the task. Rather than acting in the market, the agent must *generate* the predictive signals from which a portfolio is later built. The signals are *formulaic alphas*: short symbolic expressions whose cross-sectional value at each timestamp is treated as a forecast of future returns [9]. Each alpha is a tree of arithmetic, cross-sectional, and time-series operators over a small vocabulary of price and volume features. A portfolio is assembled from a *set* of such alphas.

Formulaic alphas remain the dominant signal-generation paradigm in low-frequency quantitative investment for two reasons that monolithic neural predictors do not satisfy. First, each formula is readable and auditable to risk committees and regulators. Second, a small set of complementary formulas degrades more gracefully than a single black box when the regime shifts. Automating their discovery is, however, a hard exploration problem under three simultaneous pressures: *quality* (the supervisory signal is the information coefficient of a finished formula on noisy data); *diversity* (what is deployed is a set covering different return sources, so a

high-quality duplicate is worthless); and *generalisation* (the training window is bounded and the test market is non-stationary).

Diagnosis. The reinforcement-learning line descended from AlphaGen [51] scores each new alpha by its marginal contribution to a running pool. This is structurally a boosting objective, and it has two pathologies, the third gap of Section 3.2. First, the reward changes every time the pool changes, so the policy is trained against a moving target. The policy never sees a stable landscape long enough to commit to coverage, which is especially damaging for exploration. Second, residual-driven boosting carries no a-priori bound on the generalisation of the resulting ensemble. The legal expression space exceeds 10^{20}, so the late-round residual maximiser is dominated by the search-space noise floor, and the retained set inherits no concentration guarantee tied to its size. Figure 6.1 contrasts the pool-dependent AlphaGen reward with the static reward AlphaQD uses.

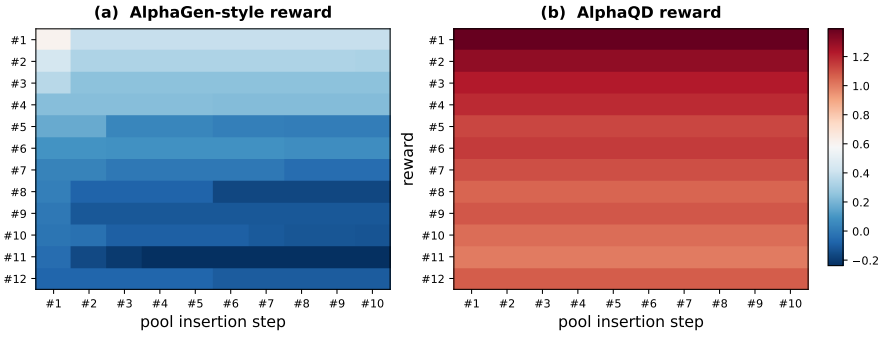


FIGURE 6.1: Per-candidate reward across pool-insertion steps under two reward functions: the AlphaGen-style pool-dependent reward (left) and the AlphaQD terminal reward (right). Rows are twelve probe alphas; columns are pool-insertion steps; colour encodes the reward (separate scale per panel). The AlphaGen reward shifts as the pool grows; the AlphaQD reward does not.

Remedy. AlphaQD removes the coupling structurally, in three stages (Figure 6.2), following the decompose–specialise–aggregate pattern with the aggregation mechanism fully detached from the reward.

1. **Efficient GFlowNet exploration.** A relational-graph GFlowNet samples symbolic expressions and is trained by trajectory balance against a terminal reward that depends *only* on the candidate's training-window information ratio, so the learning target is invariant to later archive updates.

2. **Return-source MAP-Elites archiving.** Each candidate is back-tested into a daily IC series; its information ratio scores quality, and the normalised IC series is its behaviour descriptor—an empirical proxy for the formula's return source—placing it in a PCA-discretised MAP-Elites grid.

3. **Top-k cell selection.** One high-quality representative per occupied cell is exported, and the top K cells form the deployed pool.

On full-market Chinese A-share and U.S. benchmarks against eleven baselines, AlphaQD lies in the leading signal-quality cluster on both markets, attains the highest annualised return and the strongest net Sharpe among learned formula-mining methods on A-shares, and shows the smallest seed-to-seed variance and the lowest turnover among learned pool-style methods on both markets.

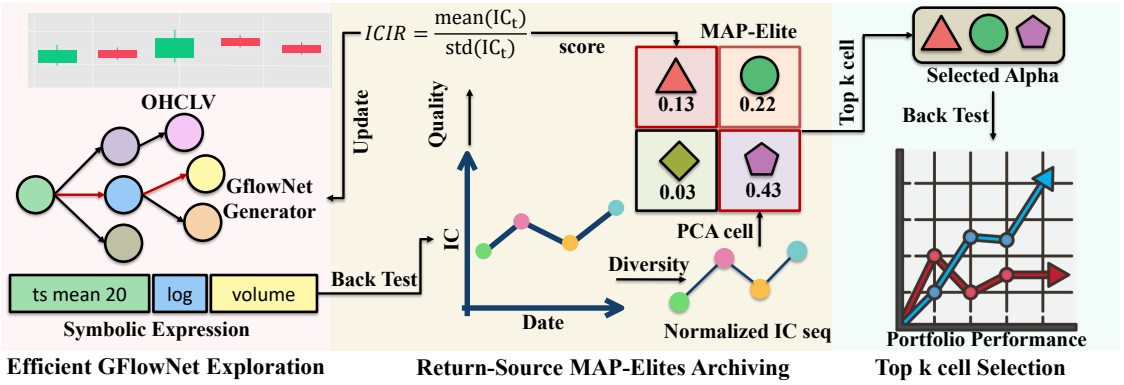


FIGURE 6.2: The three stages of AlphaQD. *Left:* a relational-graph GFlowNet samples a legal symbolic expression from raw inputs and is trained by trajectory balance. *Middle:* the candidate is back-tested into a daily IC series; the information ratio scores quality, the normalised IC series is the return-source descriptor, and the candidate enters a PCA-discretised MAP-Elites grid; the quality score is propagated back to the generator. *Right:* the top K occupied cells export their representatives as the deployed pool, evaluated by a shared ridge back-test.

6.2 Problem Formulation

Let T be the number of trading days in the training window, N the cross-sectional universe size, and $X \in \mathbb{R}^{T\times N\times d}$ the per-asset feature tensor (open, high, low, close, volume, VWAP, and short-window returns). A formulaic alpha is a function $\alpha : \mathbb{R}^{T\times N\times d} \to \mathbb{R}^{T\times N}$ realised as a reverse-Polish-notation (RPN) token sequence over a typed grammar $\mathcal{G}$, with an action mask rejecting any prefix that cannot complete

into a legal formula. Given the h-day forward return $r_{t,i}^{(h)} = \text{close}_{t+h,i}/\text{close}_{t,i} - 1$ (we use $h = 20$), the daily information coefficient and information ratio are

$$\text{IC}_t(\alpha) = \text{corr}(\alpha_t, r_t^{(h)}), \qquad \text{ICIR}(\alpha) = \overline{\text{IC}(\alpha)} \,/\, \hat{\sigma}(\text{IC}(\alpha)). \tag{6.1}$$

Mining goal. Let $\mathcal{A}$ be the set of legal formulas. The output is a bounded set $\mathcal{P} \subset \mathcal{A}$ maximising the predictive power of a downstream combination of its members on held-out data. The goal is therefore not the per-formula maximiser $\arg\max_{\alpha \in \mathcal{A}} \text{ICIR}(\alpha)$, which a single high-ICIR formula would solve, but the joint problem

$$\arg\max_{\mathcal{P} \subset \mathcal{A}} \; \mathbb{E}_{\text{test}}[\text{IC}(c(\mathcal{P}))] \quad \text{subject to} \quad |\mathcal{P}| \le K, \tag{6.2}$$

where $c(\,\cdot\,)$ is a fixed downstream combiner. This imposes two structural requirements: each retained alpha must be of high quality, and members must capture different *return sources* so their combination generalises. We use the centred, unit-normalised daily IC series as the empirical proxy for the return source carried by α.

Contrast with AlphaGen. AlphaGen approximates (6.2) by a boosting relaxation. A pool Pool_t is grown one element at a time, with the per-step terminal reward the correlation-penalised marginal contribution

$$R_t(\alpha) = \text{ICIR}(\alpha) - \lambda \max_{\alpha_j \in \text{Pool}_t} \big|\text{corr}(\text{IC}(\alpha), \text{IC}(\alpha_j))\big|. \tag{6.3}$$

Every pool update changes $R_t(\,\cdot\,)$, so the policy is trained against a moving target.

Static formulation. We retain the per-formula MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma = 1)$, in which s_t is a partial RPN prefix and a_t the next token under the legal-action mask, but we replace (6.3) with a terminal reward that depends only on the candidate and the fixed training window. Diversity is moved out of the reward and into a separate selection rule over a behaviour descriptor of α. The reward function then does not change as the archive grows, restoring stationarity; and the selection rule operates over a bounded number of behaviour cells, yielding a generalisation-complexity term that scales with the size of the retained set rather than with $|\mathcal{A}|$ (Section 6.4).

6.3 Method

The three stages exchange candidates and scalar scores but never a pool-dependent reshape of the reward. Algorithm 7 ties them together.

6.3.1 Efficient GFlowNet Exploration

Search surface. A state s_t is the partial RPN prefix and an action a_t the next token; a legal-action mask, derived from stack depth, type consistency, window-parameter constraints, and a terminator rule, confines the search to syntactically and dimensionally valid expressions. Each prefix is materialised as a typed directed graph—nodes tagged by operator, operand, constant, and window type, edges encoding operator-to-argument relations—and encoded by a two-layer relational graph convolutional network [72] of hidden width 128. The pooled node embeddings feed two heads, $\log P_F(\cdot \mid s_t)$ over legal next tokens and $\log P_B(\cdot \mid s_{t+1})$ over legal predecessors, and a scalar $\log Z$.

Terminal reward. For a completed expression α, AlphaQD evaluates its factor values on the training window, cross-sectionally winsorises and rank-normalises by date, computes $\mathrm{IC}(\alpha)$ and $\mathrm{ICIR}(\alpha)$ via (6.1), and sets

$$R(\alpha) = \max\bigl\{\exp(\beta_q\,\mathrm{ICIR}(\alpha)),\, \exp(-10)\bigr\}, \tag{6.4}$$

with quality temperature β_q and an exponential floor for unevaluable candidates. Equation (6.4) reads neither the archive nor any pool statistic—not coverage, novelty, cell identity, incumbent quality, or acceptance status—so for a fixed training window and fixed β_q the reward of a candidate is invariant to later archive updates.

Trajectory-balance training. The generator is trained by trajectory balance [17, 18]: for a terminal trajectory τ with formula $\alpha(\tau)$,

$$\mathcal{L}_{\mathrm{TB}}(\tau) = \left(\log Z + \sum_{t=0}^{T-1} \log P_F(a_t \mid s_t) - \sum_{t=0}^{T-1} \log P_B(s_t \mid s_{t+1}) - \log R(\alpha(\tau))\right)^2. \tag{6.5}$$

At the optimum, the probability of sampling a terminal expression under P_F is proportional to $R(\alpha)$, so the policy assigns mass to every high-reward mode rather than collapsing onto the single maximiser; under a fixed budget, this mode-covering sampling is the "efficient" part of the stage [17]. Failed evaluations receive the floor reward and still contribute a finite gradient, so the policy learns to avoid them without a separate invalid-expression branch.

6.3.2 Return-Source MAP-Elites Archiving

Quality and behaviour descriptor. The IC series $\mathrm{IC}(\alpha) \in \mathbb{R}^T$ supplies both the quality score $\mathrm{ICIR}(\alpha)$ and a behaviour descriptor that summarises *when* the signal is strong. AlphaQD centres and unit-normalises the IC series,

$$\mathbf{r}(\alpha) = \frac{\mathbf{x}(\alpha) - \overline{\mathbf{x}(\alpha)}\mathbf{1}}{\|\mathbf{x}(\alpha) - \overline{\mathbf{x}(\alpha)}\mathbf{1}\|_2 + \varepsilon}, \qquad \mathbf{x}(\alpha) = \mathrm{IC}(\alpha), \tag{6.6}$$

removing level and scale (absorbed into ICIR) and keeping only the relative temporal pattern. By construction $\mathbf{r}(\alpha)$ is invariant to any positive affine transform of the IC series, so it isolates which dates the formula explains, not how strong its edge is; we treat it as the empirical proxy for the formula's return source. The shape is projected by an incremental PCA basis W (canonical dimension $d_b = 2$) to $\mathbf{z}(\alpha) = W^\top \mathbf{r}(\alpha)$, then discretised by per-axis quantile bin edges into a $B \times B$ grid ($B = 10$, so 100 cells), giving the cell index $c(\alpha)$.

Per-cell ranking and update. Within each cell, elites are ranked by two terms: predictive quality (the ICIR) and a *perturbation fidelity score* (PFS) that jitters each integer window parameter by $\pm 10\%$ over five perturbations and measures the normalised agreement between the perturbed and unperturbed IC series, capturing whether quality survives small parameter changes. A candidate with non-positive ICIR is not offered for insertion. Otherwise, if its cell holds fewer than κ elites (canonical $\kappa = 3$) it enters directly; if the cell is full it enters only if it Pareto-dominates an existing elite in the $(\mathrm{ICIR}, \mathrm{PFS})$ plane or strictly improves the cell's combined score, evicting one elite to maintain the cap. The bounded quota prevents dense regions from absorbing the archive, and the Pareto front retains both the strongest and the most robust formulas per return source.

6.3.3 Top-k Cell Selection

For each occupied cell, the representative is the elite with the highest training-window ICIR (ties broken by PFS), which keeps representatives comparable across cells. Let $K^\star = \min(10, \max(5\kappa, 5))$ ($K^\star = 10$ under $\kappa = 3$, matching the pool-style baselines). AlphaQD ranks occupied cells by representative ICIR and exports the top $K^\star$ as the pool $\mathcal{P}^\star$, which is fed to a shared ridge combiner [73] fitted on the training window. Because the per-cell representative inherits the positive-ICIR admission rule and the cells partition formulas by behaviour, the exported pool satisfies the structural premises (positive standalone edges, bounded pairwise correlation) under which the ensemble bound of Section 6.4 applies.

Algorithm 7 AlphaQD: three-stage training and export

Initialise the RGCN encoder, $\log P_F$, $\log P_B$, $\log Z$, an empty MAP-Elites archive, and an empty PCA buffer.
for episode $= 1, \ldots, E$ **do**
  Stage 1: sample a batch of terminal trajectories under P_F and the legal-action mask.
  for each terminal trajectory τ with formula α **do**
    Evaluate α on the training window; compute IC(α), ICIR(α), and $R(\alpha)$ via (6.4).
    Stage 2: update the PCA buffer with $\mathbf{r}(\alpha)$ from (6.6); refit W.
    if ICIR(α) > 0 **then**
      Compute PFS via $\pm 10\%$ window-parameter perturbations.
      Insert α into cell $c(\alpha) = \text{discretize}(W^\top \mathbf{r}(\alpha))$ under the cell update rule.
    end if
    Accumulate $\mathcal{L}_{\text{TB}}(\tau)$ from (6.5).
  end for
  Update $\{\log P_F, \log P_B, \log Z\}$ on the accumulated loss.
end for
Stage 3: take the highest-ICIR elite per cell as its representative; rank cells by representative ICIR; export the top $K^\star$ as $\mathcal{P}^\star$ and fit the shared ridge combiner.

6.4 Generalisation Properties

The staging is not merely convenient; it is generalisation-favouring. This section gives the formal results, with proofs inline. Two recurring assumptions are stated by name. **(A1)** *Sign-symmetric warm-up shape distribution*: the warm-up sample

$\{\mathbf{r}(\alpha_i)\}_{i=1}^{M}$ has zero mean and its second moment equals that of its sign-augmented version $\{\mathbf{r}(\alpha_i), -\mathbf{r}(\alpha_i)\}$. A formula and its sign-flip are both legal under the same mask and produce IC series differing only by a global sign, so the warm-up buffer is approximately closed under negation. **(A2)** *Positive ridge weights on retained orbit pairs*: the shared ridge combiner assigns strictly positive weight to both representatives of an orbit when both are exported, which holds in the empirical regime where ℓ_2-regularised regression on positively-edged factors places weight on both inputs.

6.4.1 Decoupling Quality from Shape

The first result formalises why decoupling quality from the shape descriptor is sound.

Lemma 6.1 (Affine invariance of the shape descriptor)**.** *For any candidate α and any positive affine transform $\mathbf{y} = a\,\mathbf{x}(\alpha) + b\mathbf{1}$ with $a > 0$, the normalised shape vector built from $\mathbf{y}$ equals $\mathbf{r}(\alpha)$.*

Proof. The mean satisfies $\overline{\mathbf{y}} = a\,\overline{\mathbf{x}(\alpha)} + b$, so $\mathbf{y} - \overline{\mathbf{y}}\mathbf{1} = a(\mathbf{x}(\alpha) - \overline{\mathbf{x}(\alpha)}\mathbf{1})$, and normalising by the ℓ_2 norm cancels $a > 0$, giving $\mathbf{r}(\alpha)$. □

Thus the descriptor encodes only which dates were strong; the mean and scale are absorbed into the quality score. Two alphas with the same temporal pattern but different overall edge occupy the same cell rather than leaking into different ones.

6.4.2 Reward Stationarity

The second result is the structural reason the static reward survives even archive-aware exploration extensions.

Theorem 6.2 (Stationary conditional reward)**.** *Let the per-alpha reward be $R_q(\alpha) = \exp(\beta_q\,\mathrm{ICIR}(\alpha))$. For any fixed target orbit $\mathcal{O}^\star$ and any kernel $K : \mathcal{O} \times \mathcal{O} \to \mathbb{R}_{\geq 0}$ independent of the archive state and training step, the conditional reward $R_{\mathcal{O}^\star}(\alpha) = R_q(\alpha)\,K(\mathcal{O}(c(\alpha)), \mathcal{O}^\star)$ is a stationary terminal reward.*

Proof. For each fixed $\mathcal{O}^\star$, $R_{\mathcal{O}^\star}(\,\cdot\,)$ depends on α only through $\mathrm{ICIR}(\alpha)$ on the fixed training window and through the fixed PCA basis and bin edges entering $c(\,\cdot\,)$. The kernel K is fixed and $\mathcal{O}^\star$ is a parameter, not a function of training-time state. The reward therefore has no time-dependent component. A scheduler that varies $\mathcal{O}^\star$ across episodes changes only the per-episode prior over conditions, never the conditional reward itself. □

The implementation reported here uses the unconditional reward ($K \equiv 1$), so the property holds a fortiori. The conditional form documents compatibility with future descriptor-biased exploration extensions.

6.4.3 Search Complexity and the Boosting Counter-Example

The next two results contrast AlphaQD's quality-diversity ensemble with boosting-style residual fitting.

Theorem 6.3 (Residual-edge noise floor)**.** *Let $\mathcal{F} = \{\alpha_1, \ldots, \alpha_M\}$ be a finite candidate set. After $t-1$ boosting rounds with residual r_{t-1}, the empirical residual-edge maximiser is $\alpha_t = \arg\max_{\alpha\in\mathcal{F}} \widehat{\langle \alpha, r_{t-1}\rangle}$. Let $\gamma_t(\alpha) = \mathbb{E}[\langle \alpha, r_{t-1}\rangle]$ denote the population residual edge, $\Delta_t = \sup_\alpha \gamma_t(\alpha)$, and $\eta_M = \sigma\sqrt{2\log(2M/\delta)/n_{\mathrm{eff}}}$ a sub-Gaussian deviation. Then with probability at least $1-\delta$, $\gamma_t(\alpha_t) \geq \Delta_t - 2\eta_M$. When $\Delta_t \leq 2\eta_M$, the bound is non-positive.*

Proof. Uniform sub-Gaussian concentration on the finite class $\mathcal{F}$ gives $\sup_\alpha |\hat{\gamma}_t(\alpha) - \gamma_t(\alpha)| \leq \eta_M$ with probability at least $1-\delta$. Writing $\alpha_t^\star = \arg\max_\alpha \gamma_t(\alpha)$, $\gamma_t(\alpha_t) \geq \hat{\gamma}_t(\alpha_t) - \eta_M \geq \hat{\gamma}_t(\alpha_t^\star) - \eta_M \geq \gamma_t(\alpha_t^\star) - 2\eta_M = \Delta_t - 2\eta_M$, where the middle step uses the empirical optimality of α_t. □

With $|\mathcal{F}| > 10^{20}$ and n_{eff} bounded by the training-window days, the late-round residual winner is governed by the noise floor η_M rather than by population signal.

Theorem 6.4 (Search-complexity contrast)**.** *A K-step boosting procedure over $\mathcal{F}$ with $|\mathcal{F}| = M$ has an ordered-sequence hypothesis class of size at most M^K, with finite-class complexity $\mathcal{C}_{\mathrm{boost},K} = \sqrt{K\log M/n_{\mathrm{eff}}}$. A quality-diversity rule mapping candidates to at most G occupied cells and exporting one representative per cell has K-subset class of size at most $\binom{G}{K}$, with complexity $\mathcal{C}_{\mathrm{QD},K} = \sqrt{K\log(G/K)/n_{\mathrm{eff}}}$. For $G \ll M$, $\mathcal{C}_{\mathrm{QD},K} \ll \mathcal{C}_{\mathrm{boost},K}$.*

Proof. The finite-class generalisation bound scales as $\sqrt{\log|\mathcal{H}|/n_{\text{eff}}}$. Substituting $|\mathcal{H}_{\text{boost},K}| \leq M^K$ gives $\log|\mathcal{H}_{\text{boost},K}| \leq K \log M$. Substituting $|\mathcal{H}_{\text{QD},K}| \leq \binom{G}{K}$ and $\log\binom{G}{K} \leq K\log(eG/K)$ gives the QD scaling. □

In the reported grid $G \leq 100$ while $|\mathcal{A}| > 10^{20}$, so the gap spans many orders of magnitude.

6.4.4 Per-Cell Selection Bias

When a cell aggregates several candidates and the elite is selected by maximum empirical ICIR, the chosen elite's empirical ICIR is upward-biased relative to its population value—the formula-mining analogue of a multiple-testing winner's curse. The bounded per-cell quota controls this bias.

Theorem 6.5 (Per-cell selection bias under a bounded quota)**.** *Fix an occupied cell holding at most κ admitted elites at selection time, each with population quality θ_i and training-window estimate $\hat{\theta}_i = \theta_i + \varepsilon_i$ with sub-Gaussian ε_i of effective sample size n_{eff}. With probability at least $1-\delta$, the selection bias of the per-cell representative on raw ICIR satisfies $\max_{i\leq\kappa}(\hat{\theta}_i - \theta_i) \leq \sigma\sqrt{2\log(\kappa/\delta)/n_{\text{eff}}}$. If the ranking uses the PFS-augmented score that averages the ICIR with m independent perturbations, σ is replaced by $\sigma/\sqrt{1+m}$.*

Proof. A union bound on the κ sub-Gaussian tails gives $\mathbb{P}(\max_{i\leq\kappa}\varepsilon_i > t) \leq \kappa\exp(-n_{\text{eff}}t^2/(2\sigma^2))$; setting the right-hand side to δ yields $t = \sigma\sqrt{2\log(\kappa/\delta)/n_{\text{eff}}}$. Averaging $1+m$ independent sub-Gaussian estimates with parameter σ produces a sub-Gaussian estimate with parameter $\sigma/\sqrt{1+m}$. □

Under the canonical $\kappa = 3$, $\delta = 0.05$ the bound is $\approx 2.86\,\sigma/\sqrt{n_{\text{eff}}}$. The $m = 5$ perturbations of PFS tighten it by $\sqrt{6}$ when the PFS-augmented score decides the ranking.

6.4.5 Ensemble ICIR Lower Bound

The final result lower-bounds the quality of the exported ensemble.

Theorem 6.6 (Ensemble ICIR lower bound)**.** *If the export retains K positive-ICIR elites with daily IC variables $X_1, \ldots, X_K$ satisfying $\mathbb{E}[X_i] \geq \mu_{\min} > 0$, $\mathrm{Var}(X_i) \leq \sigma^2$, and $\mathrm{corr}(X_i, X_j) \leq \rho$, then the equal-weight ensemble $X_{\mathrm{ens}} = \frac{1}{K}\sum_i X_i$ obeys*

$$\mathrm{ICIR}(X_{\mathrm{ens}}) \geq \frac{\mu_{\min}\sqrt{K}}{\sigma\sqrt{1+(K-1)\rho}}. \tag{6.7}$$

Proof. By linearity, $\mathbb{E}[X_{\mathrm{ens}}] \geq \mu_{\min}$. For the variance,

$$\mathrm{Var}(X_{\mathrm{ens}}) = \tfrac{1}{K^2}\textstyle\sum_i \mathrm{Var}(X_i) + \tfrac{1}{K^2}\sum_{i\neq j}\mathrm{Cov}(X_i, X_j) \leq \tfrac{\sigma^2}{K^2}\big[K + K(K-1)\rho\big] = \tfrac{\sigma^2(1+(K-1)\rho)}{K},$$

so $\mathrm{ICIR}(X_{\mathrm{ens}}) = \mathbb{E}[X_{\mathrm{ens}}]/\sqrt{\mathrm{Var}(X_{\mathrm{ens}})} \geq \mu_{\min}\sqrt{K}/(\sigma\sqrt{1+(K-1)\rho})$. □

The bound rests on two premises that AlphaQD supplies by construction. The positive-ICIR admission rule guarantees $\mu_i > 0$. The bounded per-cell quota together with the PCA descriptor controls ρ by spreading elites across cells; Section 6.6.2 reports an empirical pairwise correlation of 0.2478. Theorem 6.3 shows that boosting-style fitting preserves neither premise, so no analogous bound is available for it.

6.5 Experimental Setup

Datasets. We evaluate on two markets so the comparison is not anchored to one regime. The Chinese A-share panel (the canonical alpha-mining setting) is the primary benchmark: full-market BaoStock daily data, 7,207 instruments over 3,963 calendar days from 2010-01-04 to 2026-04-30. The U.S. panel is an independent check with different microstructure and base rates: a Tiingo Russell-3000-style universe of 2,571 instruments over 4,109 calendar days. All methods share the same split: 2010–2021 for training, 2022 for validation, and 2023-01-01 to 2026-04-30 for testing, with forward-return horizon $h = 20$ days. The training window drives the search; the validation window fits only the shared ridge combiner; the test window is touched once per (method, seed).

Compared methods. We compare against eleven baselines in four families: supervised (Alpha101 [9], MLP [74], XGBoost [75], LightGBM [76]); genetic programming (GP single-predictor; GP with mutual-IC filter; GP without filter [47]); and generative/RL (AlphaForge [55], AlphaGen-PPO [51], AlphaQCM [52], AlphaSAGE [56]). Pool-style baselines export ten alphas each, matching AlphaQD's $K^\star = 10$.

Unified protocol. Every pool-style method, including AlphaQD, exports a fixed-size pool fed to the same closed-form ridge combiner with weights fitted on training and held fixed at test, so the table reports the quality of the discovered set rather than method-specific weighting. We report signal-layer IC, ICIR, and RankIC together with portfolio-layer annualised net Sharpe, annualised net return, maximum drawdown, and annualised turnover. The portfolio is long-only top-50 rebalanced every twenty trading days, executed at VWAP_{t+1} on A-shares and Open_{t+1} on the U.S. panel, with round-trip costs of 12.5 and 3.5 basis points respectively.

6.6 Results and Analysis

6.6.1 Main Results

Table 6.1 reports both panels under the unified-ridge protocol, aggregated over seeds $\{0, 1, 2, 12345\}$ where stochastic.

A-share findings. On the A-share panel, AlphaQD attains the highest annualised return and the highest RankIC of any method. It is second on IC (behind GP-without-filter) and second on Sharpe (behind only the fixed Alpha101 catalogue). Within the top quality cluster it also shows the smallest seed-to-seed standard deviation on Sharpe, annualised return, and RankIC. That cluster comprises GP-without-filter, AlphaForge, AlphaSAGE, and AlphaQD, whose mean ICIRs are statistically indistinguishable. The three competitors, however, do not convert signal quality into stable portfolio performance to the same extent. GP-without-filter and AlphaForge carry seed-level Sharpe deviations larger than their

TABLE 6.1: Main results on the 2023-01-01 to 2026-04-30 test window under the unified-ridge protocol. Entries are mean ± population standard deviation over seeds $\{0, 1, 2, 12345\}$ where stochastic. Portfolio metrics are net of round-trip costs on a long-only top-50 portfolio rebalanced every twenty trading days. First, second, and third per column within each panel.

Family	Method	IC↑	ICIR↑	RankIC↑	Sharpe↑	AnnRet(%)↑	MDD(%)↑	TO/yr↓
Panel A: Chinese A-share								
Formulaic	Alpha101	0.0487	0.408	0.082	0.983	19.9	−26.1	23.9
Supervised	MLP	0.0079	0.111	0.017	0.319	9.1	−49.7	12.9
	LightGBM	0.0189	0.262	0.035	0.309	10.0	−52.7	12.0
	XGBoost	0.0199	0.266	0.036	0.236	6.9	−50.6	12.4
Genetic Prog.	GP (predictor)	−0.0101	−0.082	−0.016	0.292	6.4	−40.4	3.8
	GP w/ filter	0.0652	0.507	0.105	0.663	18.6	−39.1	15.1
	GP w/o filter	0.0703	0.581	0.102	0.501	10.8	−35.7	20.2
Generative / RL	AlphaForge	0.0636	0.553	0.086	0.534	14.4	−37.0	19.5
	AlphaGen-PPO	0.0435	0.495	0.049	0.275	4.1	−43.3	22.2
	AlphaQCM (FQF)	0.0206	0.255	0.020	0.053	1.7	−46.8	19.3
	AlphaSAGE (GNN)	0.0552	0.555	0.095	0.730	14.9	−27.4	18.3
Ours	**AlphaQD**	0.0673	0.522	0.112	0.911	24.7	−31.5	12.5
Panel B: U.S. Russell-3000-style universe								
Formulaic	Alpha101	0.0188	0.358	−0.002	1.511	38.2	−15.9	23.0
Supervised	MLP	0.0446	0.604	−0.002	1.796	99.8	−16.3	9.1
	LightGBM	0.0404	0.454	−0.009	2.090	119.1	−21.6	8.7
	XGBoost	0.0335	0.395	−0.006	2.120	129.2	−23.4	9.6
Genetic Prog.	GP (predictor)	−0.0104	−0.073	0.004	1.202	39.4	−15.2	5.7
	GP w/ filter	0.0691	0.512	−0.005	2.180	141.6	−26.7	9.5
	GP w/o filter	0.0717	0.568	−0.013	2.164	141.3	−24.7	10.2
Generative / RL	AlphaForge	0.0481	0.430	−0.008	1.856	107.0	−23.4	11.6
	AlphaGen-PPO	0.0542	0.437	−0.010	2.056	129.8	−23.3	13.8
	AlphaQCM (FQF)	0.0486	0.453	−0.002	1.942	103.8	−21.1	7.9
	AlphaSAGE (GNN)	0.0614	0.539	−0.007	2.111	131.3	−24.2	10.5
Ours	**AlphaQD**	0.0655	0.551	−0.010	2.177	133.3	−22.5	6.7

means, so any single deployment is essentially a coin flip; AlphaSAGE is stable but with annualised return roughly ten points below AlphaQD and turnover 46% higher. Among learned pool-style baselines, AlphaQD also has the lowest turnover. Its gains therefore come from converting signal quality into portfolio performance reliably, not from out-signalling the field. Figure 6.3 plots the cumulative net NAV of the five learned pool-style methods.

U.S. findings. On the U.S. panel the signal-quality columns (IC, ICIR, RankIC) are the credible discriminators. RankIC clusters around zero and Sharpe-style numbers around two for all baselines, reflecting market beta in the 2023–2026 window rather than alpha-driven return. AlphaQD ranks third on both IC and ICIR. It again shows the smallest seed deviation on every IC-family metric and the lowest turnover among learned-search methods. Critically, the IC magnitude on the U.S.

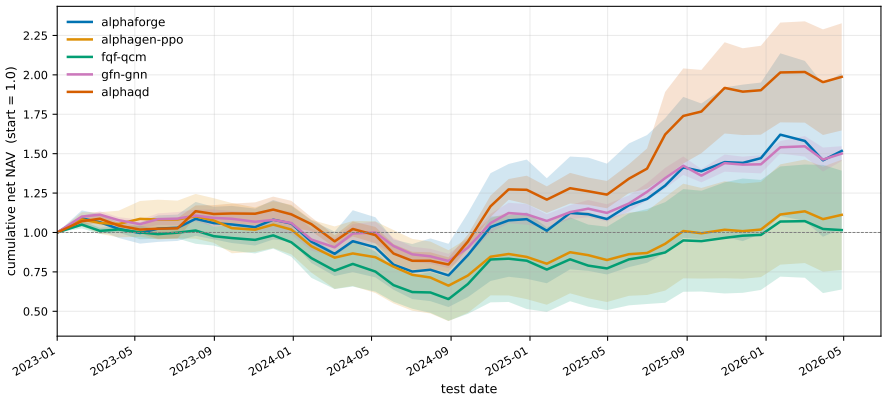


FIGURE 6.3: Cumulative net NAV (start = 1.0) on the A-share test window 2023-01-01 to 2026-04-30 for the five learned pool-style methods. Solid lines are the mean NAV across seeds $\{0, 1, 2, 12345\}$ at each rebalance date; shaded bands span ± 1 population standard deviation; the dashed line marks NAV = 1. The portfolio is the long-only top-50 rebalanced every twenty trading days at VWAP_{t+1} with 12.5 bps round-trip cost.

panel (around 0.066) matches the A-share magnitude (around 0.067). The method ordering is therefore preserved across two dissimilar markets, a cross-market stability that the boosting-style baselines do not exhibit. AlphaQCM, for instance, moves from 0.021 on A-shares to 0.049 on the U.S. panel.

6.6.2 Archive Diagnostics

On a representative training archive, AlphaQD occupies 97 of 100 behaviour cells, stores 282 elites, and reaches a best in-archive ICIR of 0.7540. The mean pairwise correlation among stored daily IC series is 0.2478. Both statistics support the descriptor's role: near-complete occupancy indicates that the normalised IC shape spreads candidates across the grid rather than collapsing into one mode, and the modest correlation indicates that retained formulas explain genuinely different return sources rather than re-encoding one signal—precisely the controlled ρ that Table 6.1's portfolio stability and Theorem 6.6 require. Figure 6.4 renders the archive.

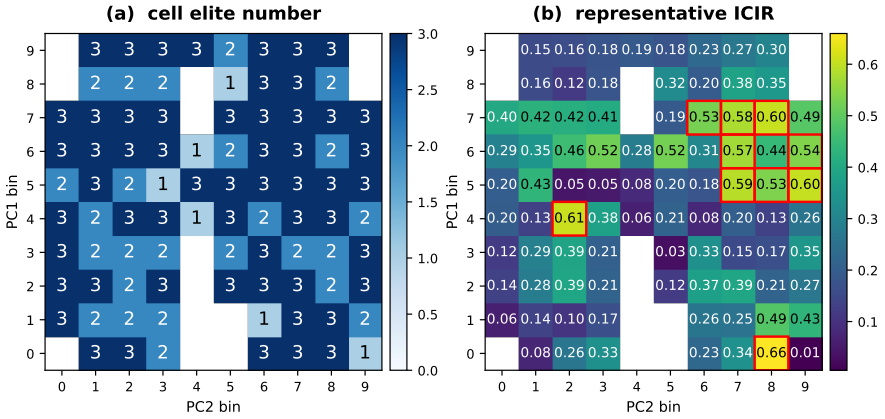


FIGURE 6.4: The AlphaQD MAP-Elites archive on the 10×10 behaviour-descriptor grid (vertical axis PC1 bin, horizontal axis PC2 bin). *(a)* number of stored elites per cell; *(b)* representative ICIR per cell, with the top-10 exported cells outlined.

6.6.3 Ablations

A grid over behaviour-descriptor dimension $d_b \in \{1, 2, 3\}$, per-cell capacity $\kappa \in \{1, 3, 5\}$, and reward temperature $\beta_q \in \{0.5, 1, 2, 5\}$ confirms the design choices. Marginal ICIR declines gently from $d_b = 1$ to $d_b = 3$, consistent with a higher-dimensional descriptor spreading candidates over more cells; the canonical $d_b = 2$ sits in the favourable region while preserving an interpretable two-axis archive. Capacity $\kappa \geq 3$ is preferred over $\kappa = 1$, supporting the in-cell Pareto front over a single elite. The reward temperature is non-monotone, peaking at $\beta_q = 2$, consistent with the boundary between coverage-preserving and quality-concentrating sampling. The full 36-configuration grid is reported in Table 6.2; within-grid noise (ICIR roughly 0.22–0.66 across cells) is larger than the marginal-mean effects, so individual grid cells should be read as samples from the AlphaQD performance distribution rather than as separated comparisons.

Computational cost. A full training run of AlphaQD on the A-share panel—2,000 GFlowNet trajectory-balance episodes with mini-batch size 64, archive maintenance, and PFS evaluation—completes in approximately five to six hours on a single NVIDIA RTX 4090 GPU. The dominant cost is the per-candidate back-test of the IC series on the full-market training window; archive insertion is $O(1)$ amortised under the per-cell quota and the PCA refit is incremental. At deployment the cost is negligible: the exported $K^\star = 10$ alphas evaluate as fixed symbolic

TABLE 6.2: AlphaQD per-configuration ablation on the A-share panel under the unified-ridge protocol (single seed, test window). Columns are descriptor dimension d_b, cell capacity κ, reward temperature β_q, signal IC, ICIR, Spearman rank-IC, and net Sharpe.

d_b	κ	β_q	IC	ICIR	RankIC	Sharpe
1	1	0.5	0.0565	0.455	0.074	0.322
1	1	1.0	0.0584	0.592	0.097	0.951
1	1	2.0	0.0772	0.647	0.127	1.027
1	1	5.0	0.0328	0.246	0.045	0.889
1	3	0.5	0.0740	0.542	0.123	1.186
1	3	1.0	0.0577	0.464	0.094	0.699
1	3	2.0	0.0757	0.630	0.106	0.540
1	3	5.0	0.0760	0.511	0.127	0.835
1	5	0.5	0.0669	0.493	0.096	0.833
1	5	1.0	0.0658	0.560	0.100	0.879
1	5	2.0	0.0683	0.664	0.107	1.010
1	5	5.0	0.0486	0.485	0.077	0.668
2	1	0.5	0.0541	0.461	0.075	0.874
2	1	1.0	0.0258	0.218	0.040	0.909
2	1	2.0	0.0724	0.662	0.106	1.175
2	1	5.0	0.0760	0.646	0.096	0.738
2	3	0.5	0.0736	0.537	0.123	0.955
2	3	1.0	0.0432	0.366	0.047	0.907
2	3	2.0	0.0724	0.541	0.112	0.718
2	3	5.0	0.0712	0.565	0.113	0.869
2	5	0.5	0.0589	0.529	0.087	0.976
2	5	1.0	0.0793	0.613	0.122	0.892
2	5	2.0	0.0375	0.511	0.087	1.005
2	5	5.0	0.0734	0.578	0.123	1.141
3	1	0.5	0.0699	0.505	0.109	0.991
3	1	1.0	0.0318	0.368	0.058	0.669
3	1	2.0	0.0502	0.516	0.080	0.644
3	1	5.0	0.0669	0.534	0.083	1.144
3	3	0.5	0.0756	0.525	0.127	1.040
3	3	1.0	0.0603	0.518	0.078	0.581
3	3	2.0	0.0554	0.616	0.085	0.957
3	3	5.0	0.0355	0.405	0.048	−0.008
3	5	0.5	0.0314	0.352	0.031	0.422
3	5	1.0	0.0741	0.562	0.130	1.025
3	5	2.0	0.0494	0.414	0.087	0.584
3	5	5.0	0.0306	0.342	0.044	0.847

expressions, and the downstream ridge combiner is a closed-form linear operation re-fit only at rebalance dates.

6.7 Chapter Summary

This chapter presented AlphaQD, a quality-diversity RL method for formulaic alpha mining. Its central choice is structural: the running pool is removed from the generator's reward, and diversity, robustness, and selection are handled by a MAP-Elites archive whose behaviour descriptor is the normalised daily IC series—an empirical proxy for a formula's return source. The training-window reward is thereby invariant to archive updates, and the export rule yields a pool whose effective complexity scales with the number of occupied cells rather than the cardinality of the formula space. On full-market A-share and U.S. benchmarks AlphaQD lies in the leading signal cluster on both markets, attains the highest annualised return and net Sharpe among learned formula-mining methods on A-shares, and exhibits the smallest seed variance and lowest turnover among learned pool-style methods. In the arc of the thesis, AlphaQD is the endpoint of the aggregation-decoupling trajectory: where EarnHFT learns its router and FineFT derives one heuristically from generative uncertainty, AlphaQD removes the aggregation mechanism from the learning reward entirely, and reaps from this decoupling both reward stationarity and a finite-sample generalisation bound that holds under explicit assumptions.

Part IV

Epilogue

Chapter 7

Conclusion and Future Work

This thesis asked what reinforcement learning can do for quantitative trading, where it fails when moved from one trading task to another, and how those failures can be repaired. It answered the question through three studies that span the trading-frequency spectrum and progress from trade execution to signal generation. This final chapter summarises the contributions, revisits the perspective that unites them, states their limitations honestly, and sketches the work they invite.

7.1 Summary of Contributions

EarnHFT (Chapter 4). For second-level high-frequency trading, the decisive obstacles are the extremely long decision horizon and regime non-stationarity. EarnHFT addressed the first with an optimal action-value teacher computed by dynamic programming over future prices. The construction is legitimate because, at high frequency, a single trader's orders do not move prices. Under that exogenous-price assumption and a fixed historical training window, the teacher preserves the optimal Bellman fixed point of the supervised update while accelerating learning empirically. EarnHFT addressed the second obstacle by training a pool of trend specialists and a minute-level router that switches among them. It exceeded the strongest of six baselines by at least 30% in profitability across four markets, and ablations confirmed that the value supervisor is the more potent half of the teacher.

FineFT (Chapter 5). For leveraged futures, leverage amplifies the aleatoric noise of the reward and makes the absence of competence-awareness fatal. FineFT trained diverse specialists efficiently through a selective-update mechanism driven by an ensemble temporal-difference matrix. As a by-product, this rule segmented the market automatically. A per-dynamic variational autoencoder then captured each specialist's capability boundary, and the router fell back to a conservative policy when the market state was out of distribution. To our knowledge, FineFT was the first to use VAEs for out-of-distribution detection in RL for quantitative trading. It reduced maximum drawdown by more than 40% relative to the runner-up among twelve baselines while remaining the most profitable learned method.

AlphaQD (Chapter 6). For formulaic alpha mining, the binding obstacle is reward design: pool-based RL miners couple the reward to an evolving pool, training the policy against a moving target and offering no finite-sample generalisation bound. AlphaQD diagnosed this coupling and removed it, training a graph-based GFlowNet against an archive-independent reward and relocating diversity into a MAP-Elites archive keyed on the normalised daily IC series, a novel return-source behaviour descriptor. The resulting selection rule admits a finite-sample ensemble bound whose complexity scales with the number of occupied cells rather than the formula space. On A-share and U.S. benchmarks AlphaQD lay in the leading signal cluster on both markets with the lowest seed variance and turnover among learned pool-style methods.

7.2 The Unifying Perspective Revisited

Across three tasks that differ in frequency, instrument, action space, and even in whether the agent acts or generates, the same response to market non-stationarity recurred: *decompose* the non-stationary market into approximately stationary sub-problems, *specialise* a learner to each, and *aggregate* the specialists at deployment. The decomposition criterion changed with the task—market trend in EarnHFT, latent dynamic in FineFT, return-source behaviour in AlphaQD—but the skeleton did not. This recurrence is not a coincidence: decomposition is the most direct way to recover, locally, the stationary-MDP assumption on which reinforcement learning depends.

The more pointed lesson concerns the aggregation mechanism. The component that composes the specialists—a router that selects one in EarnHFT and FineFT, a structural archive that combines several in AlphaQD—is also the component most easily mis-designed, because the natural impulse is to learn it jointly with the specialists or to fold its criterion into the learning reward—and either coupling re-introduces, through the back door, the non-stationarity the decomposition was meant to dissolve. The three studies trace a deliberate retreat from this coupling. EarnHFT learns its router by reinforcement learning against the trading reward—the most coupled design—so that aggregation is itself a reward-maximising policy, fitted offline to a finite span of history and inheriting the same non-stationarity as the specialists it arbitrates. FineFT replaces the learned router with a heuristic one driven by generative out-of-distribution estimates, so aggregation no longer requires policy learning. AlphaQD removes the aggregation mechanism from the learning reward altogether: the generator's reward never reads the archive, and the archive never edits the reward. The payoff of this final decoupling is concrete—reward stationarity and a finite-sample generalisation bound—and it is the single most transferable conclusion of the thesis: *in a decompose–specialise–aggregate design for a non-stationary environment, the aggregation mechanism should be kept out of the learning target.*

A second thread is methodological reuse across the chapters, which lends the thesis coherence beyond the shared skeleton. The dynamic-programming optimal action value introduced in EarnHFT reappears, with a liquidation mask, as the demonstrator and supervisor that pre-trains FineFT's ensemble. The slope-based market segmentation of EarnHFT is reused to define FineFT's dynamics. And the very limitation of EarnHFT's and FineFT's pool-and-route designs—that routing or selection, when coupled to learning, destabilises it—is what motivates AlphaQD's complete decoupling. The three studies are thus not three disconnected applications but successive iterations on one idea.

7.3 Limitations

Each study rests on assumptions that bound its claims.

EarnHFT. The optimal action-value teacher is exact only because, at high frequency, prices are assumed independent of the agent's orders; the construction would not hold for an agent large enough to move the market. EarnHFT is also a deliberately aggressive trader: its teacher conveys profit-oriented experience and little about risk, so it trails on some risk metrics. Both the teacher and the segmentation depend on hindsight price information available only on historical data.

FineFT. The dynamic-parametric MDP assumption—that dynamics evolve slowly enough that a short window is approximately stationary—can be violated by a hard regime break, which would invalidate the PCA and VAE descriptors until their buffers rotate. The conservative fallback is a heuristic stop rather than a learned policy, and the risk threshold must be tuned on validation data.

AlphaQD. Invalid or numerically unstable formulas are absorbed by the reward floor rather than by a learned invalid-expression penalty, which suffices for the markets studied but is not a structural fix. The PCA descriptor is refitted on a rolling buffer, and a hard regime break that depopulates the warm-up modes would invalidate the cell partition until the buffer rotates. The generalisation results are stated under explicit assumptions (sign-symmetric warm-up shapes, positive ridge weights on retained orbit pairs) and are not claims of global optimality.

Common scope. All three studies evaluate primarily on cryptocurrency and equity data and within high-fidelity simulators; live deployment introduces latency, partial fills, and market impact that the simulators approximate but do not reproduce exactly.

7.4 Future Work

Reflecting decoupling back onto execution. The clearest direction is to carry AlphaQD's lesson back to execution: EarnHFT's learned router and FineFT's heuristic router could be replaced by a structural, archive-style aggregation mechanism whose construction never enters the specialists' reward, testing whether alpha mining's reward-stationarity and generalisation benefits transfer to execution.

A unified cross-frequency framework. The three studies share a skeleton but not an implementation. A single framework that instantiates decompose–specialise–aggregate with task-appropriate decomposition, specialists, and decoupled aggregation across the whole frequency spectrum would turn the thesis's perspective into a reusable system.

Richer descriptors and generators. AlphaQD's behaviour descriptor could be augmented with additional economically meaningful axes, such as factor-style exposure, and its GFlowNet generator could be replaced by a language-model symbolic policy while preserving the archive-independent reward, so the archive organises diversity rather than becoming a moving component of the reward.

Risk and uncertainty as first-class objectives. FineFT separated aleatoric from epistemic uncertainty and handled each with a dedicated mechanism. Treating both as first-class objectives throughout—distributional value learning for the former and generative competence estimation for the latter, jointly optimised rather than bolted on—would unify the risk machinery developed in this thesis.

Toward deployment. Finally, closing the gap to live trading—modelling latency and market impact, and validating the decoupled-aggregation designs under realistic execution—is the necessary step from these methods to systems that operate in real markets.

Reinforcement learning is a natural language for quantitative trading, but only once the non-stationarity, risk, and reward-design pathologies of financial markets are confronted directly. This thesis confronted them for three concrete tasks and found a single, transferable principle: decompose the market, specialise to its parts, and aggregate them with a mechanism kept deliberately apart from what the specialists learn.

List of Author's Publications

The following publications and manuscripts arose from the research conducted during this doctoral candidature. The candidate is the first author of each.

Conference Proceedings

- **Molei Qin**, Shuo Sun, Wentao Zhang, Haochong Xia, Xinrun Wang, and Bo An, "EarnHFT: Efficient Hierarchical Reinforcement Learning for High Frequency Trading," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024. (Chapter 4)
- **Molei Qin**, Xinyu Cai, Yewen Li, Haochong Xia, Chuqiao Zong, Shuo Sun, Xinrun Wang, and Bo An, "FineFT: Efficient and Risk-Aware Ensemble Reinforcement Learning for Futures Trading," in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2026. (Chapter 5)

Manuscripts in Preparation

- **Molei Qin**, Meiying Zhu, Chuqiao Zong, and Bo An, "AlphaQD: Quality-Diversity Reinforcement Learning for Formulaic Alpha Mining," manuscript in preparation for submission to the *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD'27, Cycle 1)*. (Chapter 6)

Bibliography

[1] Shuo Sun, Wanqi Xue, Rundong Wang, Xu He, Junlei Zhu, Jian Li, and Bo An. Deepscalper: A risk-aware reinforcement learning framework to capture fleeting intraday trading opportunities. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 1858–1867, 2022. 4, 22, 29

[2] Antonio Briola, Jeremy Turiel, Riccardo Marcaccioli, and Tomaso Aste. Deep reinforcement learning for active high frequency trading. *arXiv preprint arXiv:2101.07107*, 2021. 22, 29, 37, 51

[3] Tian Zhu and Wei Zhu. Quantitative trading through random perturbation q-network with nonlinear transaction costs. *Stats*, 5(2):546–560, 2022. 4, 22, 37, 51

[4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013. 5, 29

[5] Chuheng Zhang, Yitong Duan, Xiaoyu Chen, Jianyu Chen, Jian Li, and Li Zhao. Towards generalizable reinforcement learning for trade execution. *arXiv preprint arXiv:2307.11685*, 2023. 5, 23, 29

[6] Ananth Madhavan. Market microstructure: A survey. *Journal of Financial Markets*, 3(3):205–258, 2000. 14

[7] Tarun Chordia, Richard Roll, and Avanidhar Subrahmanyam. Order imbalance, liquidity, and market returns. *Journal of Financial Economics*, 65(1): 111–130, 2002. 14, 37, 51

[8] Thomas Krug, Jürgen Dobaj, and Georg Macher. Enforcing network safety-margins in industrial process control using macd indicators. In *European Conference on Software Process Improvement*, pages 401–413, 2022. 15, 37, 51

[9] Zura Kakushadze. 101 formulaic alphas. *Wilmott*, 2016(84):72–81, 2016. 16, 24, 61, 72

[10] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. 17, 37, 51

[11] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016. 17, 36

[12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 17, 37, 51

[13] Will Dabney, Mark Rowland, Marc Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. 18, 52

[14] Kimin Lee, Michael Laskin, Aravind Srinivas, and Pieter Abbeel. Sunrise: A simple unified framework for ensemble learning in deep reinforcement learning. In *International Conference on Machine Learning*, pages 6131–6141, 2021. 18, 52

[15] Rundong Wang, Hongxin Wei, Bo An, Zhouyan Feng, and Jun Yao. Commission fee is not enough: A hierarchical reinforced framework for portfolio management. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 626–633, 2021. 18, 23

[16] Hui Niu, Siyuan Li, and Jian Li. Metatrader: An reinforcement learning approach integrating diverse policies for portfolio optimization. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 1573–1583, 2022. 18, 23

[17] Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow network based generative models for non-iterative diverse candidate generation. In *Advances in Neural Information Processing Systems*, volume 34, pages 27381–27394, Red Hook, NY, USA, 2021. Curran Associates, Inc. 18, 65, 66

[18] Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in GFlowNets. *Advances in Neural Information Processing Systems*, 35(1):5955–5967, 2022. 18, 65

[19] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013. 19, 50

[20] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. Finrl: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the second ACM international conference on AI in finance*, pages 1–9, 2021. 21

[21] Shuo Sun, Molei Qin, Wentao Zhang, Haochong Xia, Chuqiao Zong, Jie Ying, Yonggang Xie, Lingxuan Zhao, Xinrun Wang, and Bo An. Trademaster: A holistic quantitative trading platform empowered by reinforcement learning. In *Advances in Neural Information Processing Systems*, 2024. 21

[22] Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. Qlib: An ai-oriented quantitative investment platform. *arXiv preprint arXiv:2009.11189*, 2020. 21

[23] Shuo Sun, Molei Qin, Xinrun Wang, and Bo An. Prudex-compass: Towards systematic evaluation of reinforcement learning in financial markets. *arXiv preprint arXiv:2302.00586*, 2023. 21

[24] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep reinforcement learning for trading. *arXiv preprint arXiv:1911.10107*, 2019. 22

[25] Thibaut Théate and Damien Ernst. An application of deep reinforcement learning to algorithmic trading. *Expert Systems with Applications*, 173:114632, 2021.

[26] Yang Li, Wanshan Zheng, and Zibin Zheng. Deep robust reinforcement learning for practical algorithmic trading. *IEEE Access*, 7:108014–108022, 2019. 22

[27] Yang Liu, Qi Liu, Hongke Zhao, Zhen Pan, and Chuanren Liu. Adaptive quantitative trading: An imitative deep reinforcement learning approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2128–2135, 2020. 22

[28] Yi Ding, Weiqing Liu, Jiang Bian, Daoqiang Zhang, and Tie-Yan Liu. Investor-imitator: A framework for trading knowledge extraction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1310–1319, 2018. 22

[29] Hong-Gi Shin, Ilkyeun Ra, and Yong-Hoon Choi. A deep multimodal reinforcement learning system combined with cnn and lstm for stock trading. In *2019 International Conference on Information and Communication Technology Convergence (ICTC)*, pages 7–11, 2019. 22

[30] Lucas de Azevedo Takara, André Alves Portela Santos, Viviana Cocco Mariani, and Leandro dos Santos Coelho. Deep reinforcement learning applied to a sparse-reward trading environment with intraday data. *Available at SSRN 4411793*, 2023. 22

[31] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997. 22

[32] Peer Nagy, Jan-Peter Calliess, and Stefan Zohren. Asynchronous deep double duelling q-learning for trading-signal execution in limit order book markets. *arXiv preprint arXiv:2301.08688*, 2023. 22

[33] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017. 22

[34] Zhengyao Jiang and Jinjun Liang. Cryptocurrency portfolio management with deep reinforcement learning. In *2017 Intelligent Systems Conference (IntelliSys)*, pages 905–913, 2017. 22

[35] Zhicheng Wang, Biwei Huang, Shikui Tu, Kun Zhang, and Lei Xu. Deeptrader: A deep reinforcement learning approach for risk-return balanced portfolio management with market conditions embedding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 643–650, 2021. 22

[36] Hongyang Yang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the first ACM international conference on AI in finance*, pages 1–8, 2020. 22

[37] Yuan Gao, Ziming Gao, Yi Hu, Sifan Song, Zhengyong Jiang, and Jionglong Su. A framework of hierarchical deep q-network for portfolio management. In *ICAART (2)*, pages 132–140, 2021. 23

[38] Lijun Zha, Le Dai, Tong Xu, and Di Wu. A hierarchical reinforcement learning framework for stock selection and portfolio. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–7, 2022. 23

[39] Weiyu Si, Jinke Li, Peng Ding, and Ruonan Rao. A multi-objective deep reinforcement learning approach for stock index future's intraday trading. In *2017 10th International Symposium on Computational Intelligence and Design (ISCID)*, volume 2, pages 431–436, 2017. 23

[40] Wentao Zhang, Yilei Zhao, Shuo Sun, Jie Ying, Yonggang Xie, Zitao Song, Xinrun Wang, and Bo An. Reinforcement learning with maskable stock representation for portfolio management in customizable stock pools. In *The Web Conference 2024*, 2024. 23

[41] Jingyi Gu, Wenlu Du, AM Muntasir Rahman, and Guiling Wang. Margin trader: A reinforcement learning framework for portfolio management with margin and constraints. In *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 610–618, 2023. 23

[42] Hui Niu, Siyuan Li, Jiahao Zheng, Zhouchi Lin, Jian Li, Jian Guo, and Bo An. Imm: An imitative reinforcement learning approach with predictive representation learning for automatic market making. *arXiv preprint arXiv:2308.08918*, 2023. 23

[43] Shuo Sun, Xinrun Wang, Wanqi Xue, Xiaoxuan Lou, and Bo An. Mastering stock markets with efficient mixture of diversified trading experts. *Proceedings of the 29th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2023. 23, 35, 43, 46

[44] Tong Zhang. Regularized winnow methods. *Advances in Neural Information Processing Systems*, 13, 2000. 23, 52

[45] Molei Qin, Shuo Sun, Wentao Zhang, Haochong Xia, Xinrun Wang, and Bo An. Earnhft: Efficient hierarchical reinforcement learning for high frequency trading. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 14669–14676, 2024. 23, 43, 46, 47, 50, 52

[46] Chuqiao Zong, Chaojie Wang, Molei Qin, Lei Feng, Xinrun Wang, and Bo An. Macrohft: Memory augmented context-aware reinforcement learning on high frequency trading. *arXiv preprint arXiv:2406.14537*, 2024. 23, 43, 50, 52

[47] Trevor Stephens. gplearn: Genetic programming in python, with a scikit-learn inspired api. `https://github.com/trevorstephens/gplearn`, 2015. 24, 72

[48] Tianping Zhang, Yuanqi Li, Yifei Jin, and Jian Li. Autoalpha: an efficient hierarchical evolutionary algorithm for mining alpha factors in quantitative investment. arXiv preprint arXiv:2002.08245, 2020. 24

[49] Weizhe Ren, Yichen Qin, and Yang Li. Alpha mining and enhancing via warm start genetic programming for quantitative investment. arXiv preprint arXiv:2412.00896, 2024. 24

[50] Brenden K Petersen, Mikel Landajuela, T Nathan Mundhenk, Claudio P Santiago, Soo Kim, and Joanne T Kim. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *International Conference on Learning Representations*, pages 1–24, Online, 2021. OpenReview.net. 24

[51] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. Generating synergistic formulaic alpha collections via reinforcement learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5476–5486, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3580305.3599831. 24, 62, 72

[52] Zhoufan Zhu and Ke Zhu. Alphaqcm: Alpha discovery in finance with distributional reinforcement learning. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 1–20, Vancouver, Canada, 2025. PMLR. 24, 72

[53] Tao Ren, Ruihan Zhou, Jinyang Jiang, Jiafeng Liang, Qinghao Wang, and Yijie Peng. Riskminer: Discovering formulaic alphas via risk seeking monte carlo tree search. In *Proceedings of the 5th ACM International Conference on AI in Finance*, pages 752–760, New York, NY, USA, 2024. Association for Computing Machinery. 24

[54] Feng Xu, Yan Yin, Xinyu Zhang, Tianyuan Liu, Shengyi Jiang, and Zongzhang Zhang. Alpha^2: Discovering logical formulaic alphas using deep reinforcement learning. arXiv preprint arXiv:2406.16505, 2024. 24

[55] Hao Shi, Weili Song, Xinting Zhang, Jiahe Shi, Cuicui Luo, Xiang Ao, Hamid Arian, and Luis Angel Seco. Alphaforge: A framework to mine and dynamically combine formulaic alpha factors. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 12524–12532, Washington, DC, USA, 2025. AAAI Press. 24, 72

[56] Binqi Chen, Hongjun Ding, Ning Shen, Jinsheng Huang, Taian Guo, Luchen Liu, and Ming Zhang. Alphasage: Structure-aware alpha mining via gflownets for robust exploration. arXiv preprint arXiv:2509.25055, 2025. 24, 72

[57] Saizhuo Wang, Hang Yuan, Leon Zhou, Lionel Ni, Heung-Yeung Shum, and Jian Guo. Alpha-gpt: Human-ai interactive alpha mining for quantitative investment. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 196–206, Suzhou, China, 2025. Association for Computational Linguistics. 24

[58] Ziyi Tang, Zichen Song, Hanzhe Wang, Yuyang Ma, Tianyu Wang, Jiang Bian Sun, and Zhongyao Cong. Alphaagent: Llm-driven alpha mining with regularized exploration to counteract alpha decay. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1–12, New York, NY, USA, 2025. Association for Computing Machinery.

[59] Zhiwei Li, Ran Song, Caihong Sun, Wei Xu, Zhengtao Yu, and Ji-Rong Wen. Can large language models mine interpretable financial factors more effectively? a neural-symbolic factor mining agent model. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 1–14, Bangkok, Thailand, 2024. Association for Computational Linguistics. 24

[60] Julian Bernhard, Stefan Pollok, and Alois Knoll. Addressing inherent uncertainty: Risk-sensitive behavior generation for automated driving using distributional reinforcement learning. In *2019 IEEE Intelligent Vehicles Symposium (IV)*, pages 2148–2155, 2019. 24

[61] Annie Xie. Deep reinforcement learning amidst lifelong non-stationarity. *arXiv preprint arXiv:2006.10701*, 2020. 24, 46

[62] José Almeida and Tiago Cruz Gonçalves. A systematic literature review of investor behavior in the cryptocurrency markets. *Journal of Behavioral and Experimental Finance*, 37:100785, 2023. ISSN 2214-6350. doi: https://doi.org/10.1016/j.jbef.2022.100785. URL `https://www.sciencedirect.com/science/article/pii/S2214635022001071`. 29

[63] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018. 32

[64] Tommi Jaakkola, Michael Jordan, and Satinder Singh. Convergence of stochastic iterative dynamic programming algorithms. *Advances in Neural Information Processing Systems*, 6, 1993. 33, 34

[65] Bernard W Silverman. Spline smoothing: the equivalent variable kernel method. *The Annals of Statistics*, pages 898–916, 1984. 35

[66] Lindasalwa Muda, Mumtaj Begam, and Irraivan Elamvazuthi. Voice recognition algorithms using mel frequency cepstral coefficient (mfcc) and dynamic time warping (dtw) techniques. *arXiv preprint arXiv:1003.4083*, 2010. 35

[67] Saketh Aleti and Bruce Mizrach. Bitcoin spot and futures market microstructure. *Journal of Futures Markets*, 41(2):194–225, 2021. 43

[68] Charline Le Lan, Stephen Tu, Adam Oberman, Rishabh Agarwal, and Marc G Bellemare. On the generalization of representations in reinforcement learning. *arXiv preprint arXiv:2203.00543*, 2022. 44

[69] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(11), 2008. 44

[70] Peter J Huber. Robust estimation of a location parameter. In *Breakthroughs in Statistics: Methodology and Distribution*, pages 492–518. Springer, 1992. 47

[71] Oron Anschel, Nir Baram, and Nahum Shimkin. Averaged-dqn: Variance reduction and stabilization for deep reinforcement learning. In *International Conference on Machine Learning*, pages 176–185. PMLR, 2017. 52

[72] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference*, pages 593–607, Cham, Switzerland, 2018. Springer. 65

[73] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. 67

[74] Fionn Murtagh. Multilayer perceptrons for classification and regression. *Neurocomputing*, 2(5–6):183–197, 1991. 72

[75] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. 72

[76] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017. 72